

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Ingeniería en computación

Fundamentos Técnicos VFX

Profesor: Ing. Carlos Aldair Román Balbuena

Semestre: 2026-2

Proyecto Final: Manual Técnico.

Grupo: 03

Alumno: 318245362

Manual Técnico (Español)	6
1. Introducción	6
1.1 Propósito del Documento	6
1.2 Alcance del Sistema	6
1.3 Limitantes	6
1.4 Requerimientos Técnicos del Dispositivo	7
1.5 Stack Tecnológico	8
2. Estado del Arte: RPG por Turnos en Tiempo Real.....	8
2.1 Contexto Actual	8
2.2 Tecnologías Comparativas	9
2.3 Posicionamiento del Proyecto	9
3. Justificación de Tecnologías	10
3.1 Unity 6 con Universal Render Pipeline.....	10
3.2 C# como Lenguaje de Programación	10
3.3 Shaders HLSL Personalizados	11
3.4 Shader Graph para Efectos VFX.....	11
3.5 Blender	11
3.6 Mixamo	12
3.7 GIMP	12
4. Metodología de Desarrollo	12
4.1 Metodología Iterativo Incremental	12
4.2 Fases del Proyecto	13
4.3 Diagrama de Gantt	14
5. Arquitectura y Especificaciones	15
5.1 Arquitectura del Sistema	15
5.2 Sistema de Datos: ScriptableObjects	16
5.3 Flujo del Sistema de Combate	16
5.4 Sistema de Iluminación	16

5.5 Sistema de Materiales	17
6. Análisis Técnico de Implementación	17
6.1 Shader Personalizado	17
6.2 Efectos Visuales (VFX)	19
6.3 Iluminación Técnica	20
6.4 Postprocesado	21
6.5 Optimización y Rendimiento	22
Static Batching	22
GPU Instancing	22
6.6 Integración de Sistemas	23
6.7 Sistema de Animaciones	23
6.8 Sistema de Audio	24
7. Gestión de Assets y Créditos	24
8. Análisis de Costos	26
8.1 Costos de Mano de Obra	26
8.2 Licencias de Software	26
8.3 Hardware y Depreciación	27
8.4 Costos de Operación	27
8.5 Resumen de Costos	27
9. Diccionario Técnico	28
9.1 Sistema Central	28
9.2 Datos (ScriptableObjects)	29
9.3 Sistema de Combate	30
9.4 Sistema de Exploración	32
9.5 HUD / Interfaz	33
9.6 VFX	34
10. Resolución de Problemas	35
11. Diagramas de flujo	36
12. Conclusiones	37

Technical Manual (English)	37
1. Introduction	37
1.1 Document Purpose	37
1.2 System Scope	38
1.3 Limitations	38
1.4 Device Technical Requirements	39
1.5 Technology Stack	39
2. State of the Art: Turn-Based RPGs	40
2.1 Current Context	40
2.2 Comparative Technologies	40
2.3 Project Positioning	41
3. Technology Justification	41
3.1 Unity 6 with Universal Render Pipeline	41
3.2 C# as Programming Language	41
3.3 Custom HLSL Shaders	42
3.4 Shader Graph for VFX Effects	42
3.5 Blender	42
3.6 Mixamo	43
3.7 GIMP	43
4. Development Methodology	43
4.1 Iterative Incremental Methodology	43
4.2 Project Phases	44
4.3 Gantt Diagram	45
5. Architecture and Specifications	46
5.1 System Architecture	46
5.2 Data System: ScriptableObjects	46
5.3 Combat System Flow	47
5.4 Lighting System	47
5.5 Materials System	48

6. Technical Implementation Analysis.....	48
6.1 Custom Shader.....	48
6.2 Visual Effects (VFX)	49
6.3 Technical Lighting.....	50
6.4 Post-Processing	51
6.5 Optimization and Performance.....	52
Static Batching.....	52
GPU Instancing	52
6.6 System Integration	53
6.7 Animation System	54
6.8 Audio System.....	54
7. Asset Management and Credits	54
8. Cost Analysis	56
8.1 Labor Costs	56
8.2 Software Licenses	56
8.3 Hardware and Depreciation	57
8.4 Operating Costs	57
8.5 Cost Summary	57
9. Technical Dictionary	57
9.1 Core System	57
9.2 Data (ScriptableObjects)	59
9.3 Combat System	60
9.4 Exploration System.....	62
9.5 HUD / Interface	63
9.6 VFX	64
10. Troubleshooting.....	65
11. Flow Diagrams.....	66
12. Conclusions.....	67
13. Bibliografía / Bibliography	67

Manual Técnico (Español).

1. Introducción

1.1 Propósito del Documento

Beyond the Firmament es un prototipo de RPG por turnos construido sobre Unity 6 URP con un pipeline visual personalizado: shaders HLSL toon, efectos de partículas integrados al sistema de combate, iluminación dinámica y postprocesado de pantalla completa. Este manual documenta la arquitectura, implementación y justificación técnica de cada subsistema.

1.2 Alcance del Sistema

Beyond the Firmament cubre un ciclo completo de juego desde el menú principal hasta el combate por turnos: exploración de una mazmorra con interacción de objetos, transición a escenas de combate, resolución de encuentros contra enemigos con habilidades diferenciadas, y retorno al estado previo al combate. El prototipo incluye dos escenas funcionales (DungeonExploration y Combat_01), un elenco de personajes definidos mediante ScriptableObjects, un sistema de habilidades con lógica de área y precisión, inventario básico con objetos consumibles, y persistencia de estado entre escenas. Quedan fuera del alcance de esta versión la progresión de nivel persistente, múltiples dungeons y el sistema de diálogos.

1.3 Limitantes

El proyecto corresponde a un prototipo en fase alpha. Las limitantes se dividen en dos categorías:

Técnicas

- El pipeline está optimizado exclusivamente para Windows 64-bit con DirectX 11/12. No existe soporte validado para Linux, macOS ni plataformas móviles.

- El rendimiento del postprocesado (SSAO, Bloom) degrada proporcionalmente con la resolución. A partir de 1440p el framerate puede caer por debajo de 60 FPS en hardware de gama media.
- El sistema de guardado utiliza PlayerPrefs, lo que lo hace inadecuado para múltiples slots de guardado o datos de tamaño significativo.
- Los shaders personalizados son exclusivos del pipeline URP Forward. No son compatibles con Built-in RP ni HDRP sin reescritura.
- No hay soporte para gamepad ni opciones de accesibilidad (remapeo de controles, modos de daltonismo).

De alcance

- Sin progresión de nivel persistente entre sesiones.
- Sin dungeons adicionales ni generación procedural de mapas.
- Sin sistema de diálogos.
- Cobertura parcial de efectos de sonido.

1.4 Requerimientos Técnicos del Dispositivo

Componente	Mínimo	Recomendado
Sistema Operativo	Windows 10 (64-bit)	Windows 11 (64-bit)
Procesador (CPU)	Intel Core i5 8ª gen / AMD Ryzen 5 3600	Intel Core i7 10ª gen / AMD Ryzen 7 5800X
Memoria RAM	8 GB DDR4	16 GB DDR4/DDR5
Tarjeta Gráfica (GPU)	NVIDIA GTX 1060 / AMD RX 580	NVIDIA RTX 2060 / AMD RX 6600 XT
API Gráfica	DirectX 11 / Vulkan	DirectX 12 / Vulkan
Almacenamiento	2 GB libres (SSD o HDD)	2 GB libres en SSD NVMe
Resolución	1280×720 HD	1920×1080 Full HD

Nota: El proyecto fue desarrollado y optimizado para resolución 1920×1080. Resoluciones superiores (1440p, 4K) incrementan significativamente la carga de GPU en el pipeline de postprocesado, especialmente en el efecto de SSAO y Bloom.

1.5 Stack Tecnológico

Componente	Tecnología	Versión	Propósito
Motor de Juego	Unity	6.x (6000.0.x)	Desarrollo principal y pipeline de renderizado
Lenguaje de Programación	C#	.NET 6+	Lógica de juego, UI, sistemas de combate
Pipeline de Render	Universal Render Pipeline (URP)	17.x	Renderizado optimizado para PC/Mobile
Shaders de Entorno	HLSL personalizado	Shader Model 4.5	Estilo toon para geometría de escenario
Shaders de VFX	Shader Graph (URP)	17.x	Efectos visuales de habilidades de combate
Modelado 3D	Blender	4.x	Modificación y retopología de assets externos
Animaciones	Mixamo (Adobe)	Web	Animaciones de personaje Rhaegal (rigging automático)
IDE	JetBrains Rider / Visual Studio	2022	Entorno de desarrollo C#

2. Estado del Arte: RPG por Turnos en Tiempo Real

2.1 Contexto Actual

Los RPG por turnos experimentaron un resurgimiento significativo entre 2020 y 2025. Títulos como Sea of Stars (Sabotage Studio, 2023) lograron recaudar más de \$1.6 millones USD en Kickstarter, demostrando la viabilidad comercial del género con equipos pequeños. Octopath Traveler II (Square Enix, 2023) demostró que la combinación de estética retro con iluminación técnica avanzada genera alta aceptación crítica. En el ámbito independiente, Cassette Beasts (Bytten Studio, 2023) implementó un sistema de

fusión de criaturas que sirvió de referencia para sistemas de combate emergentes. La tendencia actual privilegia estéticas artísticas fuertes (low-poly, pixel art, cel-shading) sobre el hiperrealismo en proyectos de escala media o pequeña.

2.2 Tecnologías Comparativas

Para el desarrollo de un RPG por turnos con efectos visuales integrados, se evaluaron tres plataformas principales:

Unity 6 con URP: Ofrece el ecosistema más maduro para proyectos con VFX basados en partículas, Shader Graph integrado, compatibilidad nativa con C# y una comunidad activa de 3.9 millones de desarrolladores. El URP proporciona un pipeline de renderizado eficiente para hardware de gama media. La licencia Personal es gratuita para proyectos académicos. Su principal limitación es el soporte inferior a Lumen (Unreal) para iluminación global en tiempo real.

Unreal Engine 5: Superiora en fotorrealismo mediante Lumen y Nanite, con Blueprints como sistema de scripting visual. Sin embargo, la curva de aprendizaje es pronunciada para equipos sin experiencia previa, el ejecutable mínimo supera los 500 MB, y los requisitos de hardware son más elevados. El enfoque de Unreal favorece proyectos AAA con alto presupuesto de producción.

Godot 4: Motor open-source con lenguaje GDScript similar a Python. Apropiado para prototipos 2D y proyectos educativos ligeros, pero el ecosistema de VFX es significativamente inferior a Unity y Unreal. La falta de Shader Graph nativo dificulta la creación de efectos visuales artísticos complejos.

2.3 Posicionamiento del Proyecto

Beyond the Firmament se posiciona como prototipo técnico-académico orientado a demostrar la integración coherente de múltiples técnicas de VFX dentro de un sistema de juego funcional. La elección de Unity 6 con URP responde a la necesidad de equilibrar calidad visual artística (toon shading), facilidad de implementación de VFX (Shader Graph) y rendimiento en hardware de gama media. El estilo Dark Fantasy Toon, caracterizado por geometría low-poly con sombreado cel-shading y paletas de color

saturadas, permite lograr una identidad visual distintiva sin requerir presupuesto de assets AAA.

3. Justificación de Tecnologías

3.1 Unity 6 con Universal Render Pipeline

Unity 6 fue seleccionado como motor principal por las siguientes razones técnicas: (1) el Universal Render Pipeline proporciona un sistema de Global Volume para postprocesado totalmente integrado con el pipeline de renderizado, eliminando la necesidad de implementaciones custom de efectos de pantalla completa; (2) la arquitectura de ScriptableObjects de Unity permite definir datos de personajes y habilidades de forma modular y serializable, facilitando la iteración de balance sin recompilación; (3) Shader Graph ofrece un editor visual de nodos que permite a artistas técnicos crear efectos visuales complejos (distorsión UV, efectos de disolución, máscaras procedurales) sin conocimiento avanzado de HLSL; (4) el sistema de Particle System (Shuriken) proporciona integración nativa con componentes de juego, permitiendo activar efectos mediante eventos sin código adicional; (5) el Asset Store ofrece assets de bajo costo conformes con las restricciones de la asignatura (Inferno World Free, Low Poly Dungeons Lite), reduciendo tiempo de producción de assets base.

3.2 C# como Lenguaje de Programación

C# fue elegido por su tipado estático que reduce errores en tiempo de ejecución en sistemas complejos como el CombatManager, su integración nativa con el motor Unity mediante MonoBehaviour y ScriptableObject, el soporte completo de patrones de diseño (Singleton para GameManager, Observer para eventos de combate), y la disponibilidad de herramientas de análisis estático (JetBrains Rider con RiderFlow). La ausencia de gestión manual de memoria (garbage collection de .NET) reduce la complejidad del desarrollo frente a C++, con impacto mínimo en rendimiento para un prototipo de RPG por turnos donde los tiempos de renderizado por frame no son críticos entre turnos.

3.3 Shaders HLSL Personalizados

Los shaders HLSL personalizados (Toon_Environment, Toon_General, Toon_Rhaegal) fueron escritos directamente en código en lugar de usar Shader Graph por las siguientes razones técnicas: (1) mayor control sobre el pipeline de vértices para implementar técnicas como vertex extrusion para outlines; (2) acceso directo a directivas de compilación como `#pragma multi_compile_instancing`, necesaria para habilitar GPU Instancing en materiales de geometría estática; (3) posibilidad de implementar variantes de shader condicionales mediante keywords sin overhead visual del editor de Shader Graph; (4) el modelo de iluminación toon (ramp-based diffuse con bandas discretas) requiere acceso directo al valor `NdotL` que resulta más limpio en HLSL que en nodos de Shader Graph. Los shaders soportan el pipeline Forward Rendering de URP y son compatibles con el sistema de sombras estándar de Unity.

3.4 Shader Graph para Efectos VFX

El Shader Graph de URP fue utilizado específicamente para los efectos visuales de habilidades de combate (SG_Acometida, SG_CaprichoFloral y otros) por razones complementarias a los shaders HLSL: (1) la velocidad de iteración visual es superior para efectos que requieren ajuste frecuente de parámetros de distorsión UV, emisión y transparencia; (2) la integración nativa con el sistema de Particle System permite que los materiales de partículas utilicen propiedades expuestas del Shader Graph como variables de control; (3) los efectos de VFX no requieren GPU Instancing ni las directivas de compilación especiales de los shaders de geometría, por lo que no existen ventajas técnicas de escribirlos en HLSL puro para este caso de uso.

3.5 Blender

Blender fue seleccionado como herramienta de modelado y animación por ser software libre sin costo de licencia, lo que lo hace sostenible para un proyecto de desarrollo independiente. Ofrece un pipeline completo en una sola herramienta: modelado poligonal, unwrapping UV, rigging, skinning y edición de animaciones, eliminando la necesidad de transferir assets entre aplicaciones distintas. Su flujo de trabajo con modificadores no destructivos permite iterar sobre la geometría sin perder el estado base del modelo. La exportación nativa a FBX garantiza compatibilidad directa con Unity sin

pérdida de jerarquías, normales ni datos de animación. En el proyecto se utilizó tanto para el modelado de personajes originales como para la modificación de assets externos: retopología, ajuste de escala, corrección de UVs y creación de la animación de apertura del cofre.

3.6 Mixamo

Las animaciones del personaje protagonista Rhaegal fueron obtenidas a través de Mixamo (Adobe), que proporciona un sistema de rigging automático y una biblioteca de animaciones de uso libre para proyectos educativos y comerciales. El proceso incluyó la subida del modelo del personaje para rigging automático, selección de animaciones de idle, caminar, atacar y recibir daño, y exportación en formato FBX para importación en Unity. Las animaciones fueron integradas mediante un Animator Controller con parámetros booleanos y triggers que responden a los eventos del CombatManager.

3.7 GIMP

GIMP fue utilizado para el procesamiento y edición de texturas del proyecto. Ofrece las capacidades esenciales para un pipeline de texturas 2D: capas, máscaras, canales alpha y ajustes de color no destructivos, sin costo de licencia. Su soporte de formatos estándar (PNG, TGA, PSD) mantiene compatibilidad directa con el importador de texturas de Unity. Se empleó principalmente para ajuste de paleta de color de texturas externas, generación de mapas de transparencia y corrección de seamless tiling en materiales de entorno.

4. Metodología de Desarrollo

4.1 Metodología Iterativo Incremental

El proyecto siguió una metodología Iterativo Incremental, dividiendo el desarrollo en ciclos cortos donde cada iteración entrega un incremento funcional verificable. Esta estrategia fue seleccionada porque los componentes del proyecto tienen naturalezas diferenciadas — arte 3D, programación de sistemas y pipeline visual — que pueden desarrollarse en paralelo parcial sin bloquear otras áreas. Los requisitos visuales (shaders, iluminación, postprocesado) requieren ajuste frecuente al observar resultados

en ejecución, por lo que las iteraciones cortas proporcionan retroalimentación visual rápida. Los componentes de mayor riesgo técnico, como el desarrollo de shaders personalizados y la lógica de combate, se abordaron en fases intermedias para tener margen de corrección antes del cierre.

4.2 Fases del Proyecto

El desarrollo se estructuró en cuatro fases ejecutadas del 8 de febrero al 25 de mayo de 2026.

Fase 1 — Pre-producción (08/02/2026 – 21/03/2026)

Establecimiento de los fundamentos conceptuales, visuales y de diseño del proyecto. Se realizó la investigación de referencias de estilo visual para definir la dirección artística Dark Fantasy Toon, seguida de la documentación de objetivo, alcance y limitantes del prototipo. Paralelamente se ejecutó el modelado de los personajes principales en Blender y el diseño de las escenas de exploración y combate junto con el prototipado del HUD.

Fase 2 — Implementación (22/03/2026 – 11/04/2026)

Integración de los assets producidos al proyecto Unity y desarrollo de los sistemas de juego centrales. Se exportaron modelos y texturas desde Blender con configuración de UVs y materiales base, se implementó la lógica de exploración de mazmorra, y se desarrolló el sistema de combate por turnos con su menú de acciones.

Fase 3 — Assets y Rigging (12/04/2026 – 09/05/2026)

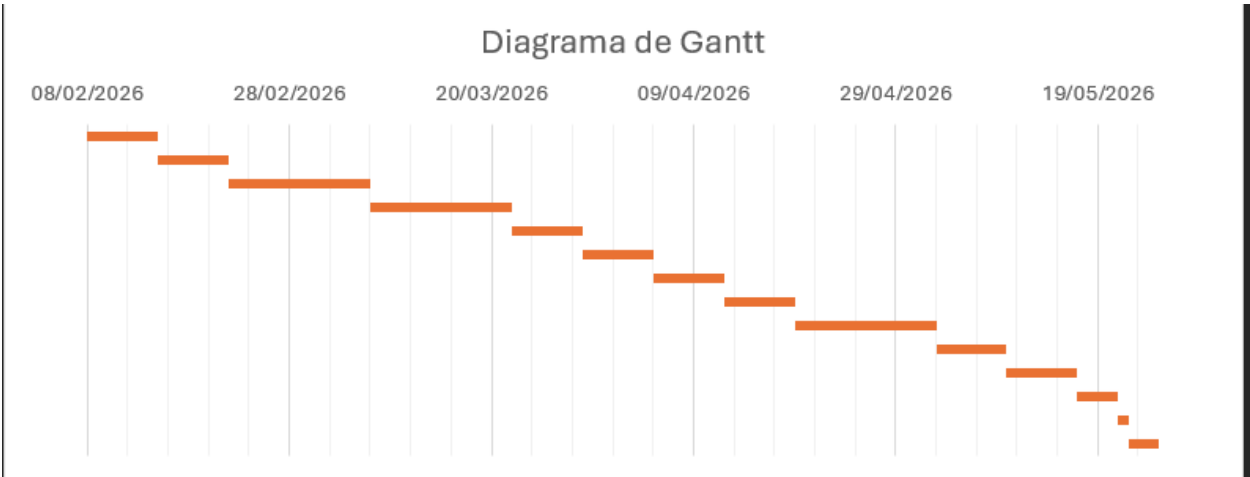
Producción de los elementos visuales avanzados del proyecto. Se generaron las animaciones de ciclo (idle y poses de combate) mediante Mixamo y ajuste en Unity, se desarrollaron los shaders HLSL toon personalizados para el pipeline URP, y se implementaron los sistemas de partículas y efectos de Shader Graph para las habilidades de combate.

Fase 4 — Cierre (10/05/2026 – 25/05/2026)

Refinamiento final del prototipo y entrega. Se configuró el postprocesado completo (Tonemapping, Bloom, Viñeta, Ajustes de Color, SSAO), se aplicaron las técnicas de

optimización de rendimiento (Static Batching, GPU Instancing) y se realizaron pruebas de rendimiento. La documentación técnica y de usuario se redactó en paralelo a las pruebas, con entrega final el 21 de mayo y preparación de presentación hasta el 25 de mayo.

4.3 Diagrama de Gantt



Fase	Actividad	Fecha Inicio	Duración (Días)	Fecha Fin
Pre-producción	Investigación, Referencias y Estilo Visual	08/02/2026	7	14/02/2026
	Planeación: Objetivo, Alcance y Limitantes	15/02/2026	7	21/02/2026
	Diseño de Personajes 3D	22/02/2026	14	07/03/2026
	Diseño de Escenario y Prototipado de HUD	08/03/2026	14	21/03/2026
Implementación	Exportación modelos y texturas	22/03/2026	7	28/03/2026
	Programación de Exploración	29/03/2026	7	04/04/2026
	Programación de Lógica de Turnos y Menú	05/04/2026	7	11/04/2026
Assets Rigging y	Animación de Ciclos (Idle y Poses)	12/04/2026	7	18/04/2026

	Desarrollo de Shaders	19/04/2026	14	02/05/2026
	Implementación de VFX y Partículas	03/05/2026	7	09/05/2026
Cierre	Iluminación, Post-procesado y Pulido	10/05/2026	7	16/05/2026
	Pruebas de Rendimiento y Documentación	17/05/2026	4	20/05/2026
	Entrega Final	21/05/2026	1	21/05/2026
	Preparación de Presentación	22/05/2026	4	25/05/2026

5. Arquitectura y Especificaciones

5.1 Arquitectura del Sistema

El sistema está organizado en cuatro capas funcionales:

Capa de Datos (ScriptableObjects): CharacterData y Magic son ScriptableObjects que almacenan estadísticas base de personajes y parámetros de habilidades de forma serializable e independiente del código. Esta arquitectura permite modificar el balance del juego sin recompilar, facilita la prueba de configuraciones mediante el Inspector de Unity, y separa los datos de diseño de la lógica de programación.

Capa de Lógica de Juego: CombatManager orquesta el flujo de combate por turnos, gestionando el orden de acción, la resolución de ataques (verificación de precisión mediante Random.Range, cálculo de daño considerando defensa del objetivo, aplicación de área de efecto cuando esArea=true). GameManager persiste entre escenas mediante DontDestroyOnLoad e implementa el patrón Singleton para acceso global.

Capa de Presentación Visual: CombatAnimator coordina la reproducción de animaciones de personaje (Animator Controller con Mixamo clips), la instanciación de efectos de partículas (targetEffectPrefab referenciado en cada Magic ScriptableObject) y la reproducción de clips de audio (audioClip del Magic) sincronizados con los tiempos de lanzamiento (tiempoLanzamiento, effectDuration).

Capa de Pipeline de Renderizado: URP con ForwardRenderer gestiona el renderizado de escenas. Los shaders HLSL personalizados se ejecutan en el Forward Pass. El Global Volume aplica efectos de postprocesado en pantalla completa. El PC_Renderer.asset incluye el RendererFeature de SSAO (Screen Space Ambient Occlusion).

5.2 Sistema de Datos: ScriptableObjects

El proyecto utiliza dos tipos de ScriptableObjects definidos en código C# (CharacterData y Magic):

CharacterData contiene: nombre, descripcion, ataqueFisicoBase, poderMagico, defensaBase, vidaBase, velocidadBase, evasionBase, precisionBase, ataquesPorTurno, y un arreglo de referencias a Magic ScriptableObjects. Esta estructura permite definir personajes completos como activos de Unity editables en el Inspector.

Magic contiene: nombre, descripcion, magicType (físico/mágico), esArea (bool para afectar a todos los aliados/enemigos), valor (daño/curación base), precision (probabilidad de impacto, 0-100), usosMaximos, castClip (AnimationClip), tiempoLanzamiento, projectilePrefab, targetEffectPrefab (VFX Prefab), effectDuration y audioClip.

5.3 Flujo del Sistema de Combate

El flujo de combate sigue el siguiente ciclo por turno: (1) CombatManager determina el personaje activo según velocidadBase; (2) SubMenu presenta las opciones disponibles (Atacar, Magia, Defensa, Objeto); (3) el jugador selecciona acción y objetivo; (4) CombatManager llama a CombatAnimator.PlayAbility(magic, caster, target); (5) CombatAnimator reproduce el castClip en el Animator del caster durante tiempoLanzamiento segundos; (6) instancia el targetEffectPrefab en la posición del objetivo; (7) espera effectDuration segundos para que el VFX complete su ciclo; (8) CombatManager aplica el valor de daño/curación ajustado por estadísticas; (9) verifica condición de victoria o derrota; (10) pasa el turno al siguiente personaje activo.

5.4 Sistema de Iluminación

La escena utiliza dos tipos de fuentes de luz en modo Realtime. Una Directional Light actúa como luz principal de la escena con color blanco cálido, intensidad 1 y rotación (X:

50°, Y: -30°) que simula una fuente de luz cenital lateral. Tiene Soft Shadows habilitadas con strength 1. El color de sombra global está configurado en #6B7AA0, un azul-violáceo frío que genera un contraste complementario con los tonos cálidos de las luces puntuales, técnica estándar en iluminación artística para aumentar la sensación de profundidad. Las Point Lights posicionadas en antorchas y candelabros de la mazmorra complementan la Directional Light con el componente FlickerAnimator, que anima su intensidad de forma aleatoria suavizada para simular llama orgánica. El Environment Lighting y las Environment Reflections toman como fuente el Skybox activo de la escena.

5.5 Sistema de Materiales

Los materiales del proyecto se dividen en dos categorías según su complejidad. Las superficies de gran área visible, como el piso y el techo, utilizan juegos de texturas completos con albedo, normal map, mapa de desplazamiento, AO y specular, aportando detalle de alta frecuencia donde la extensión de la superficie lo justifica visualmente. El resto de los materiales, que corresponde a la mayoría de los assets de la mazmorra y los personajes, aplican el shader toon personalizado con paletas de color planas sin textura adicional. Esta decisión es coherente con el estilo low-poly del proyecto: la geometría facetada y el sombreado cel-shading hacen que las texturas fotorrealistas sean visualmente inconsistentes con la dirección artística, por lo que el color plano es una elección deliberada de producción.

6. Análisis Técnico de Implementación

Esta sección detalla la implementación técnica de cada subsistema del proyecto, incluyendo las decisiones de diseño, los parámetros configurados y las alternativas evaluadas.

6.1 Shader Personalizado

Tres shaders HLSL personalizados para URP.

Se desarrollaron tres shaders HLSL escritos en código (no Shader Graph) para el pipeline Forward de URP: Toon_Environment.shader para geometría de entorno (paredes, columnas, pisos, objetos estáticos), Toon_General.shader para objetos con propiedades

de material variables, y Toon_Rhaegal.shader para el personaje protagonista con características específicas de sombreado.

El modelo de iluminación implementado es toon/cel-shading basado en el producto punto normalizado entre el vector de luz y la normal de superficie (NdotL). En lugar de aplicar el valor de NdotL directamente (Lambertian diffuse continuo), se utiliza una función de cuantización que agrupa el resultado en bandas discretas, produciendo el característico sombreado de dibujo animado con transiciones abruptas entre zonas iluminadas y en sombra.

```
float NdotL = dot(normalize(normalWS), normalize(lightDir));  
  
float toon = smoothstep(0.0, 0.02, NdotL);  
  
float3 diffuse = toon * lightColor * albedo;  
  
float spec = pow(max(0, dot(halfDir, normalWS)), 32.0);  
  
float toonSpec = step(0.5, spec);  
  
float3 specular = toonSpec * lightColor * 0.5;
```

Fragmento representativo del cálculo de iluminación toon (pseudocódigo HLSL):

Soporte de GPU Instancing

Durante la fase de optimización se identificó que los shaders HLSL personalizados no incluían las directivas de compilación necesarias para GPU Instancing. Se añadieron las siguientes modificaciones al Toon_Environment.shader:

```
#pragma multi_compile_instancing  
  
UNITY_VERTEX_INPUT_INSTANCE_ID  
  
UNITY_SETUP_INSTANCE_ID(IN);
```

Estas adiciones permiten que Unity agrupe instancias del mismo mesh+material en un solo draw call cuando GPU Instancing está habilitado en el material, resultando en una reducción de 18 draw calls adicionales sobre el Static Batching ya aplicado.

Justificación de la técnica

El cel-shading fue seleccionado por coherencia con la dirección artística Dark Fantasy Toon del proyecto. Este estilo reduce la dependencia de mapas de normales de alta resolución, aprovechando la geometría de bajo conteo de polígonos al hacer que las facetas planas sean parte del estilo visual. La paleta de colores saturada del proyecto se integra naturalmente con el modelo de iluminación toon, que tiende a exagerar las transiciones de color de manera favorable para la legibilidad visual.

6.2 Efectos Visuales (VFX)

Sistemas de partículas y shaders de VFX para habilidades de combate.

Los efectos visuales de las habilidades de combate se implementaron mediante dos mecanismos complementarios: sistemas de partículas (Particle System de Unity/Shuriken) y materiales de Shader Graph asignados a los emisores de partículas.

Cada habilidad (Magic ScriptableObject) referencia un `targetEffectPrefab` que contiene un Particle System configurado con el material de Shader Graph correspondiente. Cuando `CombatAnimator.PlayAbility()` es invocado, el prefab se instancia en la posición del objetivo, se reproduce durante `effectDuration` segundos y se destruye automáticamente. Esta arquitectura desacopla la definición del efecto (ScriptableObject) de su presentación visual (Prefab), permitiendo añadir nuevas habilidades sin modificar código de combate.

Los Shader Graph desarrollados para VFX incluyen parámetros expuestos como Dissolve Amount, Emission Intensity y UV Scroll Speed, lo que permite que el sistema de partículas interpole estos valores durante el ciclo de vida de las partículas mediante el módulo `Renderer` y `Custom Vertex Streams`.

Justificación de la técnica

El sistema de partículas de Unity fue seleccionado sobre VFX Graph porque tiene soporte nativo en el URP Forward renderer sin configuración adicional, es compatible con todos los objetivos de plataforma del proyecto, y su comportamiento es determinista en el contexto de un juego por turnos donde los efectos deben completarse en tiempos sincronizados con la lógica de combate. VFX Graph, al estar basado en cómputo en

GPU, hubiera requerido ajustes adicionales de compatibilidad con el Forward renderer del URP en Unity 6.

6.3 Iluminación Técnica

Directional Light como fuente principal y Point Lights dinámicas con FlickerAnimator en antorchas y candelabros.

La escena DungeonExploration utiliza iluminación dinámica en tiempo real compuesta por dos tipos de fuente. Una Directional Light actúa como luz principal con color blanco cálido, intensidad 1 y ángulo (X: 50°, Y: -30°) que simula iluminación cenital lateral con Soft Shadows habilitadas. El color de sombra global está configurado en #6B7AA0, un azul-violáceo frío que contrasta con los tonos cálidos de las luces puntuales, generando profundidad mediante contraste complementario.

Las Point Lights posicionadas en antorchas y candelabros tienen asignado el componente FlickerAnimator, que anima su intensidad siguiendo una curva aleatoria suavizada para simular el comportamiento orgánico de una llama. Sus colores en el rango naranja-ámbar refuerzan la atmósfera de mazmorra y contrastan con el color de sombra global del entorno.

Decisión sobre iluminación estática vs. dinámica

Se evaluó Light Baking (Lightmapping) para mejorar el rendimiento, pero fue descartado por las siguientes razones: los Point Lights con FlickerAnimator son dinámicos y no pueden incluirse en un lightmap estático sin producir inconsistencias visuales entre la luz base baked y el parpadeo realtime; el personaje requeriría Light Probes para recibir la iluminación del lightmap, añadiendo complejidad de configuración; y la iluminación completamente dinámica produce resultados visuales satisfactorios con el shader toon sin el overhead del proceso de bake.

Decisión sobre Occlusion Culling

Se evaluó Occlusion Culling mediante el sistema integrado de Unity, pero fue descartado porque la escena DungeonExploration tiene geometría de caja abierta con visibilidad directa a todos los objetos desde cualquier posición del personaje. En esta configuración

no se generarían ocluidores efectivos, y el costo de procesamiento del bake y la verificación en tiempo de ejecución excedería cualquier beneficio de rendimiento.

6.4 Postprocesado

URP Global Volume con cinco efectos configurados.

El postprocesado se implementa mediante un Global Volume de URP que referencia el perfil `SampleSceneProfile.asset`, compartido por las escenas `DungeonExploration` y `Combat_01` para garantizar coherencia visual entre ambas. El Global Volume opera en modo Global, aplicando sus efectos en todo momento sin condición de proximidad.

Tonemapping (Neutral): mapea el rango HDR del render buffer al rango LDR del display. El modo Neutral preserva la saturación de los colores del shader toon mejor que ACES (que introduce viraje cálido) o Filmic (que comprime highlights de forma no lineal), produciendo la representación más fiel a los valores de color definidos en los materiales.

Bloom (Umbral: 0.6, Intensidad: 1.0, Scatter: 0.4): genera un halo de luz suave alrededor de los píxeles que superan el umbral de luminosidad. Con umbral 0.6 el efecto se activa principalmente en fuentes de luz directas y highlights especulares del shader toon. El Scatter 0.4 controla la extensión del halo sin hacerlo invasivo.

Vignette (Intensidad: 0.35, Smoothness: 0.4, Rounded: true): oscurece los bordes de la pantalla concentrando la atención en el centro del encuadre. El modo Rounded produce una viñeta ópticamente más natural que la variante rectangular predeterminada.

Color Adjustments (Contraste: +15, Saturación: +20): el incremento de contraste complementa la cuantización del shader toon enfatizando la diferencia entre zonas iluminadas y en sombra. El aumento de saturación intensifica los naranjas cálidos de las antorchas y los azules fríos de las sombras, acentuando la paleta dark fantasy sin producir colores que rompan la coherencia artística.

Screen Space Ambient Occlusion (Intensidad: 0.4): implementado como `RendererFeature` en `PC_Renderer.asset`. Calcula oclusión ambiental en espacio de pantalla, oscureciendo esquinas, grietas y uniones entre superficies. Con intensidad 0.4

añade profundidad sin producir halos artefactuales en los bordes, problema frecuente en geometría low-poly cuando la intensidad supera 0.6.

6.5 Optimización y Rendimiento

Static Batching

El Static Batching agrupa en tiempo de build los meshes de objetos marcados como Static en un solo buffer de vértices combinado, eliminando el overhead de draw calls individuales por objeto.

Los objetos marcados como Static en DungeonExploration incluyen paredes, columnas, pisos, techos, escaleras, pedestales y props decorativos no animados. Las luces con FlickerAnimator y los objetos recogibles fueron excluidos por ser dinámicos en tiempo de ejecución.

Resultado medido en el Stats Panel de Unity: 170 batches antes → 105 batches después, reducción de 65 batches (38.2%), verificado mediante Frame Debugger.

GPU Instancing

El GPU Instancing permite renderizar múltiples instancias del mismo mesh+material en un solo draw call, pasando las transformaciones individuales como un buffer de datos en GPU.

Se habilitó Enable GPU Instancing en los 18 materiales de la escena que utilizan Toon_Environment. Adicionalmente fue necesario añadir soporte explícito en el shader HLSL, ya que a diferencia de Shader Graph, los shaders escritos en código requieren las directivas de compilación manualmente: `#pragma multi_compile_instancing`, `UNITY_VERTEX_INPUT_INSTANCE_ID` en el struct `Attributes`, y `UNITY_SETUP_INSTANCE_ID(IN)` como primera instrucción del vertex shader.

Técnicas evaluadas y descartadas

Object Pooling: descartado porque la frecuencia de instanciación de VFX en un RPG por turnos (1-2 efectos cada 2-5 segundos) está muy por debajo del umbral donde el pooling genera beneficio medible.

Level of Detail (LOD): descartado porque la cámara fija isométrica mantiene todos los objetos a distancias similares, sin variación de tamaño en pantalla que justifique múltiples niveles de detalle.

Lightmapping: descartado por incompatibilidad con las fuentes de luz dinámicas `FlickerAnimator`, detallada en la sección 6.3.

6.6 Integración de Sistemas

Integración coherente entre sistema de datos, lógica de combate, animaciones, VFX y audio.

La integración de sistemas se manifiesta en la cadena completa de ejecución de una habilidad: el `Magic ScriptableObject` contiene referencias a todos los recursos necesarios para su ejecución (`AnimationClip`, `VFX Prefab`, `AudioClip`) sin acoplamiento a clases específicas. `CombatManager` consume los datos del `ScriptableObject` para las decisiones de lógica (daño, área de efecto, precisión) y delega la presentación a `CombatAnimator`, que coordina la secuencia temporal de animación, VFX y audio.

El `GameManager` implementa el patrón Singleton con `DontDestroyOnLoad`, persistiendo a través de las escenas (Menu > `DungeonExploration` > `Combat_01` > Menu) y manteniendo coherente el estado del juego entre transiciones. El `SaveSystem` utiliza `PlayerPrefs` para serializar el estado sin dependencias externas.

Coherencia del pipeline visual

El tiempoLanzamiento del `Magic ScriptableObject` define cuándo se instancia el VFX, sincronizado con el frame culminante de la animación de ataque. El `effectDuration` define cuándo `CombatManager` aplica el daño, sincronizado con el clímax visual del efecto. Esta coordinación temporal produce una experiencia cohesiva donde presentación visual y mecánica de juego están alineadas.

6.7 Sistema de Animaciones

Las animaciones de los personajes se gestionan mediante `Animator Controllers` en Unity con clips importados desde Mixamo en formato FBX. Cada personaje tiene definidos estados de animación (`Idle`, `Walk`, `Attack`, `TakeDamage`) conectados mediante

transiciones condicionadas por parámetros booleanos y triggers. CombatAnimator es el componente responsable de activar estos parámetros en respuesta a los eventos del CombatManager, garantizando que la animación de ataque se ejecute antes de la instanciación del VFX y que la animación de daño responda a la resolución del combate. Los tiempos de transición entre estados están ajustados para minimizar cortes visuales abruptos sin introducir delays perceptibles en la respuesta de combate.

6.8 Sistema de Audio

El audio opera en dos niveles. La música de fondo se reproduce mediante un AudioSource en loop asignado al GameManager, lo que permite que persista entre escenas sin interrupciones durante las transiciones. Los efectos de sonido de habilidades están referenciados directamente en cada Magic ScriptableObject mediante el campo audioClip, permitiendo que CombatAnimator los reproduzca de forma sincronizada con la animación y el VFX correspondientes sin requerir lógica de audio centralizada. Los clips de SFX se reproducen en modo 2D dado que la cámara es fija y la posición del personaje en pantalla no varía lo suficiente para justificar audio posicional con atenuación espacial.

7. Gestión de Assets y Créditos

Los siguientes assets fueron utilizados en el proyecto. Todos los assets externos fueron modificados o adaptados para integrarse al pipeline visual y técnico del prototipo.

Asset	Plataforma	Autor(es)	Tipo	Modificaciones Aplicadas
SFX de combate	Freesound.org	nicksta18, eldiariosonoro_, cjosephwalker, SilverIllusionist, steprock, colorsCrimsonTea rs, gezortenplotz, NoahBangs, Coghezzi, magnuswaker	Audio (SFX)	Normalización de niveles, recorte de silencios, ajuste de formato para Unity AudioSource

BGM	peritune.co m	Asset	Peritune	Audio (Música)	Bucle configurado en Unity AudioSource, ajuste de volumen relativo a SFX
Inferno World Free	Unity Store	Asset	ithappy	Entorno 3D	Shaders reemplazados por Toon_Environment.sha der personalizado
Low Poly Dungeons Lite	Unity Store	Asset	JustCreate	Geometría de mazmorra	Retopología en Blender, shaders reemplazados, adición de animación de apertura de cofre
Low Poly RPG Bundle Part 3	Unity Store	Asset	GualtierisGG	Props y personajes	Modificación de proporciones en Blender, shaders toon aplicados, ajuste de UVs
Surreal- Skyboxes	Unity Store	Asset	igor-lir	Skybox	Selección de variante para atmósfera dark fantasy
Backgroun d de combate	Vecteezy.co m	—	—	Imagen 2D	Ajuste de saturación y contraste para coherencia con paleta del juego
Animacion es de personaje	Mixamo	—	Adobe Inc.	Animacion es FBX	Rigging automático, selección de clips idle/ataque/daño, configuración en Animator Controller
Personajes principales	Elaboración propia	—	—	Modelos 3D (Blender)	Modelado original, retopología, UV unwrapping y preparación para rigging
Motor juego de	Unity Technologies	—	—	Software	N/A

8. Análisis de Costos

8.1 Costos de Mano de Obra

El proyecto fue desarrollado por un integrante asumiendo múltiples roles técnicos. La estimación de horas se basa en las actividades registradas en el diagrama de Gantt, con una tarifa de referencia de mercado para perfil junior en desarrollo de videojuegos y arte técnico.

Rol	Horas Estimadas	Tarifa (MXN/hora)	Total (MXN)
Artista 3D (modelado, rigging, animación)	180	\$150	\$27,000
Desarrollador (C#, sistemas de juego)	160	\$150	\$24,000
Technical Artist (shaders, VFX, postprocesado)	100	\$150	\$15,000
Diseño y documentación	60	\$150	\$9,000
Total	500		\$75,000

8.2 Licencias de Software

Software	Licencia	Justificación	Costo (MXN)
Unity 6	Personal (gratuita)	La licencia Personal es gratuita para proyectos con ingresos anuales inferiores a \$200,000 USD. Un prototipo de esta escala no supera ese umbral.	\$0
Blender 4.x	Open Source (GNU GPL)	Software libre sin restricciones de uso comercial.	\$0
GIMP	Open Source (GNU GPL)	Software libre sin restricciones de uso comercial.	\$0
Mixamo	Plan gratuito (Adobe)	Gratuito para uso personal y comercial sin límite de descargas.	\$0

JetBrains Rider	Comercial individual	Durante el desarrollo se utilizó licencia estudiantil gratuita. El costo comercial de referencia es \$69 USD/año (~\$1,173 MXN al tipo de cambio actual).	\$1,173
Total			\$1,173

8.3 Hardware y Depreciación

Componente	Especificación	Precio Estimado (MXN)	Vida Útil	Depreciación Mensual	Meses de Uso	Total (MXN)
HP 15.6	Victus Ryzen 5 7525HS, 32GB DDR5, RTX 2050	\$18,999	36 meses	\$527.75	3.5	\$1,847

8.4 Costos de Operación

Concepto	Detalle	Costo (MXN)
Electricidad	500h × 100W promedio = 50 kWh × \$2.50 MXN/kWh	\$125
Internet	\$450 MXN/mes × 3.5 meses (proporcional al proyecto)	\$1,575
Total		\$1,700

8.5 Resumen de Costos

Concepto	Costo (MXN)
Mano de obra	\$75,000
Software	\$1,173
Hardware (depreciación)	\$1,847
Costos de operación	\$1,700
Total del Proyecto	\$79,720

9. Diccionario Técnico

9.1 Sistema Central

Script	Propósito	Métodos Principales	Variables Clave
AudioManager.cs	Singleton persistente. Gestiona BGM y SFX de todo el juego, cambiando la música automáticamente al detectar cambios de escena.	PlayBGM(clip), PlaySFX(clip), PlayBGMFin(), StopBGM(), PlayBoton(), PlayAtaqueBasico(), PlayEvilLaugh()	bgmSource, sfxSource, bgmMenu, bgmExploracion, bgmCombate, bgmFin, sfxAtaqueBasico, sfxRecibirDano, sfxMuerte
PartyManager.cs	Singleton persistente. Almacena y sincroniza el estado de la party entre escenas.	LoadIntoStats(partyStats[]), SaveFromStats(partyStats[])	characterDatas[], characterSaveDatas (List)
SaveSystem.cs	Gestiona la serialización y deserialización del estado del juego mediante PlayerPrefs.	Save(data), Load(), HasSave(), DeleteSave()	SAVE_KEY (constante)
ExplorationLoader.cs	Restaura la posición del jugador e inventario al cargar la escena de exploración desde un guardado existente.	Start()	playerTransform

MainMenuManager.cs	Controla los botones del menú principal. Habilita o deshabilita "Continuar" según si existe guardado.	NuevoJuego(), Continuar()	botonContinuar, escenaExploracion
PauseMenuManager.cs	Gestiona el menú de pausa en exploración. Guarda la partida al pausar.	Pausar(), Reanudar()	pauseMenu, playerTransform, pausado

9.2 Datos (ScriptableObjects)

Script	Propósito	Métodos Principales	Variables Clave
CharacterData.cs	Define las estadísticas base de un personaje y su lista de habilidades. Instanciable desde el Inspector sin código.	—	nombre, ataqueFisicoBase, poderMagico, defensaBase, vidaBase, velocidadBase, evasionBase, precisionBase, ataquesPorTurno, magias[]
Magic.cs	Define los parámetros de una habilidad: tipo, valor, precisión y referencias a recursos visuales y de audio.	—	nombre, magicType, esArea, valor, precision, usosMaximos, castClip, tiempoLanzamiento, targetEffectPrefab, effectDuration, audioClip

ItemData.cs	Define los datos de un ítem de inventario: nombre, descripción y cantidad de curación que aplica.	—	nombre, descripcion, healAmount
Item.cs	Instancia runtime de un ItemData con cantidad. Utilizado por el sistema de inventario.	Item(itemData, quantity)	ItemData, ItemQuantity, nombre, descripcion
CharacterSaveData.cs	Estructura serializable que almacena el estado persistente de un personaje entre escenas.	CharacterSaveData(characterData)	characterData, currentLv, currentXp, currentHealth, hasBeenSaved

9.3 Sistema de Combate

Script	Propósito	Métodos Principales	Variables Clave
CombatManager.cs	Orquesta el flujo de combate por turnos. Gestiona fases de selección y ejecución, y condiciones de victoria o derrota.	SelectAction(action), ElegirObjetivoAliviado(stats), EjecutarTurnos()	partyMembers[], enemy, currentState, turnData[], currentPartyIndex, esperandoObjetivo
CombatCalculator.cs	Clase estática con las fórmulas matemáticas del sistema de combate. Separada del Manager para mantener la lógica de cálculo aislada.	Precision(precision, evasion, turnData), ActionValue(emitter, receptor, turnData)	— (clase estática sin estado)
CombatAnimator.cs	Coordina la secuencia temporal de animación, VFX y audio para cada acción de combate.	Init(party, enemy), PlayAbility(magic, caster, target), PlayAttack(caster, target)	partyAnimators[], enemyAnimator, effectDuration
CharacterAnimator.cs	Envuelve el Animator de un personaje y	PlayAttackUntilLaunch(),	duracionAtaque, duracionMagia,

	expone coroutines sincronizadas con los tiempos de los clips de Mixamo.	PlayAttackRem ainder(), PlayMagicUntil Launch(magic), PlayMagicRem ainder(magic), PlayDefend(), PlayTakeHit(), PlayDie(), SetMagicClip(clip), SetWalking(boo l)	tiempoLanzamiento, puntoLanzamiento, puntoImpacto, magicPlaceholderClip Name
ActionMenu.cs	Gestiona el menú principal de acciones en combate (Atacar, Magia, Defensa, Objeto). Bloquea botones mientras se espera selección de objetivo.	AtackBtn_Click(), MagicBtn_Click (), DefenseBtn_Cli ck(), ObjectBtn_Click ()	atackBtn, defenseBtn, magicBtn, objectBtn, subMenu, combatManager
SubMenu.cs	Presenta submenús contextuales (lista de magias, lista de ítems) al seleccionar una acción en ActionMenu.	ShowMagicMenu(), ShowItemMenu (), HideSubMenu()	combatManager, magicButtons[], itemButtons[]
CharacterSelect.cs	Permite seleccionar un aliado como objetivo haciendo clic sobre su representación en pantalla.	OnPointerClick(eventData), OnPointerEnter (eventData), OnPointerExit(eventData)	misEstadisticas, combatManager, marcoVisual
CombatLoggers	Sistema de log de combate con efecto typewriter y cola de mensajes. CombatManager espera a IsIdle antes de avanzar de fase.	Log(message), LogSeleccion(nombre), LogPhase(phase)	combatDialogue, letraspOrSegundo, tiempoEntresMensajes, cola (Queue), IsIdle
EndScreen.cs	Pantalla de fin de partida (victoria o derrota) con fade y	Mostrar(victoria)	panel, mensaje, textoContinuar, escenaMenu

espera de input para regresar al menú.

9.4 Sistema de Exploración

Script	Propósito	Métodos Principales	Variables Clave
PlayerController.cs	Controla el movimiento del jugador con Rigidbody, animaciones de caminar y recogida de ítems con tecla E.	RecogerItem(item), FixedUpdate(), Update()	speed, rotationSpeed, characterAnimator, nearbyItem, intervaloPasos
CheckBounds.cs	Restringe la posición del jugador dentro de los límites del BoxCollider del suelo.	Update()	floor, offset, xMax, xMin, zMax, zMin
CameraFollow.cs	Sigue al jugador con interpolación suave y transiciona entre configuración de cámara de exploración y combate.	EventStart(), EventEnd()	offsetX, offsetY, offsetZ, velocidadMov, combatFOV, exploreRotation, combatRotation
CombatStart.cs	Gestiona la secuencia de inicio de combate al contacto con zona trigger: mueve al enemigo, dispara audio dramático y carga la escena de combate.	IniciarCombate(), SmoothLookAt(looker, target, duration)	player, enemy, enemyTarget, cameraFollow, speed, pausaDramatica, walkAnimSpeed
ChestManager.cs	Gestiona la interacción con el cofre del tesoro: activación de animación de apertura y entrega de recompensa.	OnInteract(), OpenChest(), GiveReward()	isOpen, animatorRef, reward
Inventory.cs	Singleton persistente. Gestiona el inventario global del jugador con soporte para apilado de ítems.	AddItem(item), RemoveItem(item), SearchItem(item)	ItemsList (List)

CollectableItem.cs	Componente asignado a objetos recogibles en escena. Contiene la referencia al ItemData correspondiente.	—	itemData
---------------------------	---	---	----------

9.5 HUD / Interfaz

Script	Propósito	Métodos Principales	Variables Clave
CharacterStats.cs	Gestiona las estadísticas dinámicas de un personaje en tiempo de juego: HP actual, nivel, experiencia y cálculo de stats derivadas.	TakeDamage(damage), ReceiveHeal(amount), RecalcularStats(), RefreshHealthUI(), StatLvUpdate(baseStat, nivel)	currentHealth, currentMaxHealth, currentLv, currentXp, characterData, healthBar, xpBar, StatScale
HealthBarSystem.cs	Anima suavemente la barra de vida hacia un valor objetivo usando Mathf.MoveTowards.	SetHealth(percentage)	healthBar (Image), barSpeed, nextHealth
XpBarSystem.cs	Anima suavemente la barra de experiencia. Se reinicia visualmente al subir de nivel.	SetXp(percentage), LevelUp(level)	xpBar (Image), barSpeed, nextXp
PickupNotifier.cs	Singleton que gestiona un stack de notificaciones toast al recoger ítems. Los toasts aparecen apilados y ascienden flotando.	Notify(texto)	toastPrefab, toastContainer, toastSpacing, toastDuration, activeToasts (List)
PickupToast.cs	Un toast individual de recogida. Sube flotando y se	SetText(texto), Animar(duration, risePerSecond)	canvasGroup, rectTransform

desvanece durante
su ciclo de vida.

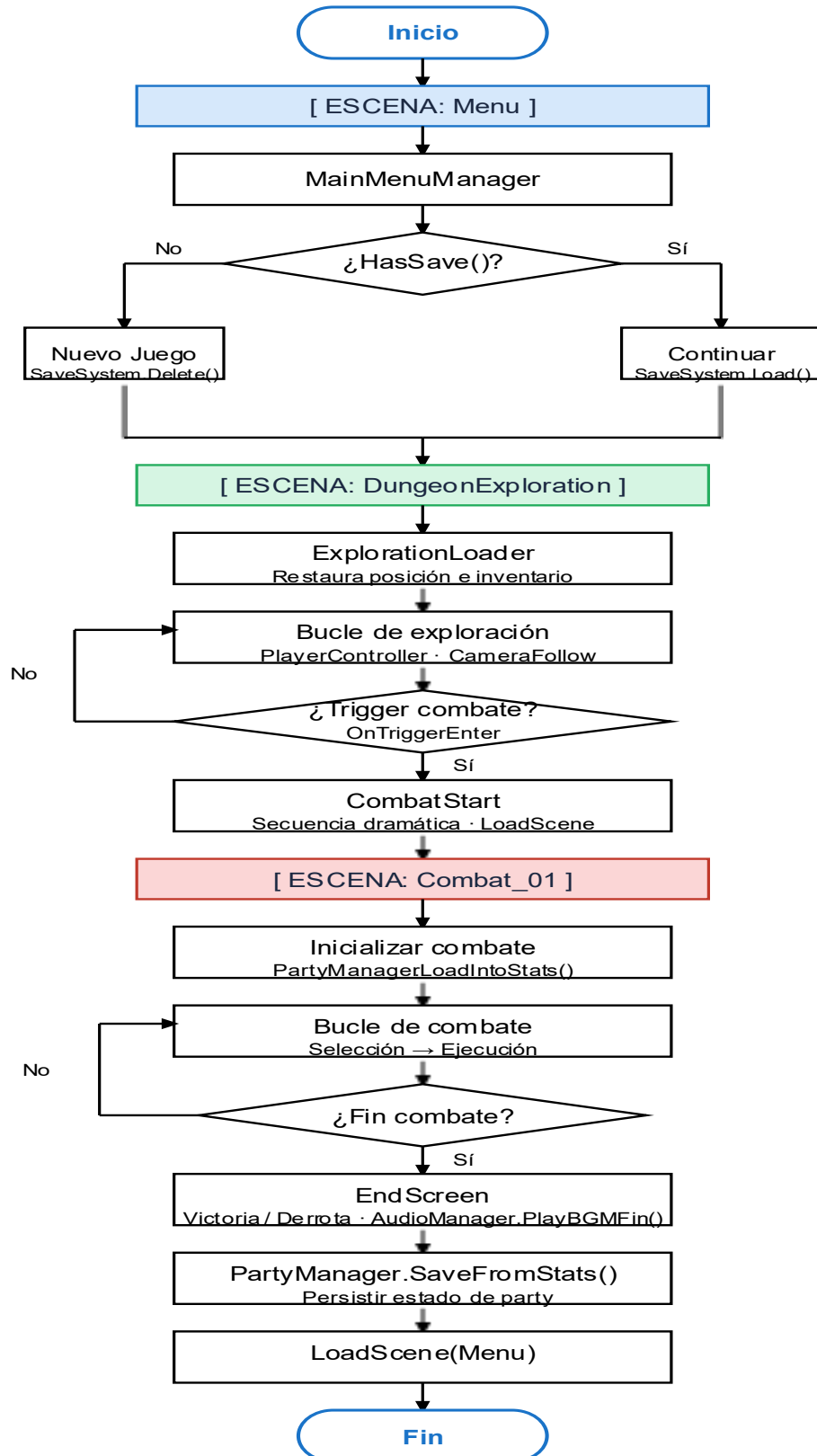
9.6 VFX

Script	Propósito	Métodos Principales	Variables Clave
FlickerAnimator.cs	Anima la intensidad de un Point Light con variación aleatoria suavizada para simular llama orgánica.	Update()	minIntensity, maxIntensity, flickerSpeed, lightRef
Billboard.cs	Rota el Transform para que siempre mire hacia la cámara. Usado en el quad de la llama de antorcha.	LateUpdate()	copyRotation
DissolveAnimator.cs	Anima la propiedad _DissolveAmount del material de un efecto VFX: fade-in al spawn y fade-out antes de destruirse.	Start(), Update()	lifetime, fadeIn, fadeOut, dissolveProperty, disappearProperty
BeamConnector.cs	Conecta visualmente dos puntos (caster y target) mediante un efecto de haz continuo.	Connect(origin, target), Disconnect()	lineRenderer, origin, target
Projectile.cs	Controla el movimiento de proyectiles instanciados por habilidades de combate con targetEffectPrefab de tipo proyectil.	Launch(target), Update()	speed, targetPosition, hasHit

10. Resolución de Problemas

Problema	Síntomas	Causa	Solución
GPU Instancing no activo tras habilitar checkbox	El Frame Debugger mostraba draw calls individuales por instancia a pesar de tener Enable GPU Instancing activado en materiales	El shader HLSL personalizado (Toon_Environment) no incluía las directivas de compilación requeridas por GPU Instancing	Añadir <code>#pragma multi_compile_instancing, UNITY_VERTEX_INPUT_INSTANCE_ID</code> en <code>Attributes</code> , y <code>UNITY_SETUP_INSTANCE_ID(IN)</code> en vertex shader
Rendimiento bajo en resolución 4K durante desarrollo	Framerate < 30 FPS en modo Play del editor con resolución de Game View en 3840×2160	El postprocesado (especialmente SSAO y Bloom) escala cuadráticamente con la resolución de pantalla	Configurar Game View a 1920×1080 durante desarrollo. La resolución de build final es independiente de la Game View del editor.

11. Diagramas de flujo



12. Conclusiones

Beyond the Firmament establece una base técnica funcional que integra de forma coherente seis subsistemas de renderizado y VFX dentro de un prototipo de RPG por turnos. El proyecto demuestra que Unity 6 con URP es una plataforma viable para producción independiente con identidad visual artística definida, permitiendo combinar shaders HLSL personalizados, sistemas de partículas, iluminación dinámica y postprocesado de pantalla completa sin comprometer la arquitectura de juego.

Las decisiones técnicas más relevantes del proyecto — iluminación completamente dinámica justificada por la incompatibilidad del baking con FlickerAnimator, descarte del Occlusion Culling por geometría de caja abierta, y la adición manual de soporte GPU Instancing en los shaders HLSL — reflejan un proceso de evaluación técnica que va más allá de la aplicación mecánica de técnicas, considerando el impacto real de cada decisión sobre la arquitectura existente y el resultado visual.

Como líneas de trabajo futuro se identifican: implementación de VFX Graph para efectos de mayor complejidad computacional, sistema de guardado basado en serialización JSON para soporte de múltiples slots, integración de animaciones faciales mediante blend shapes, y preparación del pipeline para resoluciones superiores a 1080p mediante optimización del postprocesado.

Technical Manual (English)

1. Introduction

1.1 Document Purpose

Beyond the Firmament is a turn-based RPG prototype built on Unity 6 URP with a custom visual pipeline: HLSL toon shaders, particle effects integrated into the combat system, dynamic lighting, and full-screen post-processing. This manual documents the architecture, implementation, and technical justification of each subsystem.

1.2 System Scope

Beyond the Firmament covers a complete game cycle from the main menu to turn-based combat: dungeon exploration with object interaction, transitions to combat scenes, encounter resolution against enemies with differentiated abilities, and return to the pre-combat state. The prototype includes two functional scenes (DungeonExploration and Combat_01), a roster of characters defined via ScriptableObjects, an ability system with area-of-effect and accuracy logic, basic inventory with consumable items, and cross-scene state persistence. Out of scope for this version: persistent level progression, multiple dungeons, and the dialogue system.

1.3 Limitations

The project is an alpha-phase prototype. Limitations are divided into two categories:

Technical

- The pipeline is optimized exclusively for 64-bit Windows with DirectX 11/12. No validated support exists for Linux, macOS, or mobile platforms.
- Post-processing performance (SSAO, Bloom) degrades proportionally with resolution. Above 1440p, framerate may drop below 60 FPS on mid-range hardware.
- The save system uses PlayerPrefs, making it unsuitable for multiple save slots or large data volumes.
- Custom shaders are exclusive to the URP Forward pipeline. They are not compatible with Built-in RP or HDRP without rewriting.
- There is no gamepad support or accessibility options (control remapping, color blindness modes).

Scope

- No persistent level progression between sessions.
- No additional dungeons or procedural map generation.

- No dialogue system.
- Partial sound effect coverage.

1.4 Device Technical Requirements

Component	Minimum	Recommended
Operating System	Windows 10 (64-bit)	Windows 11 (64-bit)
Processor (CPU)	Intel Core i5 8th gen / AMD Ryzen 5 3600	Intel Core i7 10th gen / AMD Ryzen 7 5800X
RAM	8 GB DDR4	16 GB DDR4/DDR5
Graphics Card (GPU)	NVIDIA GTX 1060 / AMD RX 580	NVIDIA RTX 2060 / AMD RX 6600 XT
Graphics API	DirectX 11 / Vulkan	DirectX 12 / Vulkan
Storage	2 GB free (SSD or HDD)	2 GB free on NVMe SSD
Resolution	1280×720 HD	1920×1080 Full HD

Note: The project was developed and optimized for 1920×1080 resolution. Higher resolutions (1440p, 4K) significantly increase GPU load in the post-processing pipeline, especially for the SSAO and Bloom effects.

1.5 Technology Stack

Component	Technology	Version	Purpose
Game Engine	Unity	6.x (6000.0.x)	Main development and rendering pipeline
Programming Language	C#	.NET 6+	Game logic, UI, combat systems
Render Pipeline	Universal Render Pipeline (URP)	17.x	Optimized rendering for PC/Mobile
Environment Shaders	Custom HLSL	Shader Model 4.5	Toon-style shading for scene geometry
VFX Shaders	Shader Graph (URP)	17.x	Visual effects for combat abilities
3D Modeling	Blender	4.x	Modification and retopology of external assets

Animations	Mixamo (Adobe)	Web	Rhaegal animations rigging)	character (automatic rigging)
IDE	JetBrains Rider / 2022 Visual Studio		C# environment	development

2. State of the Art: Turn-Based RPGs

2.1 Current Context

Turn-based RPGs experienced a significant resurgence between 2020 and 2025. Titles such as *Sea of Stars* (Sabotage Studio, 2023) raised over \$1.6 million USD on Kickstarter, demonstrating the commercial viability of the genre with small teams. *Octopath Traveler II* (Square Enix, 2023) proved that combining retro aesthetics with advanced technical lighting generates strong critical reception. In the indie space, *Cassette Beasts* (Bytten Studio, 2023) implemented a creature fusion system that served as a reference for emergent combat systems. The current trend favors strong artistic aesthetics (low-poly, pixel art, cel-shading) over hyper-realism in medium and small-scale projects.

2.2 Comparative Technologies

For developing a turn-based RPG with integrated visual effects, three main platforms were evaluated:

Unity 6 with URP: Offers the most mature ecosystem for particle-based VFX projects, with integrated Shader Graph, native C# compatibility, and an active community of 3.9 million developers. URP provides an efficient rendering pipeline for mid-range hardware. The Personal license is free for academic projects. Its main limitation is inferior real-time global illumination support compared to Unreal's Lumen.

Unreal Engine 5: Superior for photorealism via Lumen and Nanite, with Blueprints as a visual scripting system. However, the learning curve is steep for teams without prior experience, minimum executables exceed 500 MB, and hardware requirements are higher. Unreal's focus favors AAA projects with high production budgets.

Godot 4: Open-source engine with GDScript, a Python-like language. Suitable for 2D prototypes and lightweight educational projects, but the VFX ecosystem is significantly inferior to Unity and Unreal. The lack of native Shader Graph makes creating complex artistic visual effects more challenging.

2.3 Project Positioning

Beyond the Firmament is positioned as a technical-academic prototype focused on demonstrating the coherent integration of multiple VFX techniques within a functional game system. The choice of Unity 6 with URP reflects the need to balance artistic visual quality (toon shading), ease of VFX implementation (Shader Graph), and performance on mid-range hardware. The Dark Fantasy Toon style — characterized by low-poly geometry with cel-shading and saturated color palettes — achieves a distinctive visual identity without requiring an AAA asset budget.

3. Technology Justification

3.1 Unity 6 with Universal Render Pipeline

Unity 6 was selected as the primary engine for the following technical reasons: (1) URP provides a fully integrated Global Volume system for post-processing within the rendering pipeline, eliminating the need for custom full-screen effect implementations; (2) Unity's ScriptableObject architecture enables modular and serializable character and ability data definitions, facilitating balance iteration without recompilation; (3) Shader Graph provides a visual node editor that allows technical artists to create complex visual effects (UV distortion, dissolve effects, procedural masks) without advanced HLSL knowledge; (4) the Particle System (Shuriken) provides native integration with game components, enabling effects to be triggered via events without additional code; (5) the Asset Store offers low-cost assets compliant with project restrictions (Inferno World Free, Low Poly Dungeons Lite), reducing base asset production time.

3.2 C# as Programming Language

C# was chosen for its static typing, which reduces runtime errors in complex systems such as CombatManager; its native integration with the Unity engine via MonoBehaviour

and ScriptableObject; full support for design patterns (Singleton for GameManager, Observer for combat events); and the availability of static analysis tools (JetBrains Rider with RiderFlow). The absence of manual memory management (.NET garbage collection) reduces development complexity compared to C++, with minimal performance impact for a turn-based RPG prototype where per-frame render times are not critical between turns.

3.3 Custom HLSL Shaders

The custom HLSL shaders (Toon_Environment, Toon_General, Toon_Rhaegal) were written directly in code rather than using Shader Graph for the following technical reasons: (1) greater control over the vertex pipeline to implement techniques such as vertex extrusion for outlines; (2) direct access to compilation directives like `#pragma multi_compile_instancing`, required to enable GPU Instancing on static geometry materials; (3) the ability to implement conditional shader variants via keywords without the visual editor overhead of Shader Graph; (4) the toon lighting model (ramp-based diffuse with discrete bands) requires direct access to the NdotL value, which is cleaner in HLSL than in Shader Graph nodes. The shaders support URP's Forward Rendering pipeline and are compatible with Unity's standard shadow system.

3.4 Shader Graph for VFX Effects

URP's Shader Graph was used specifically for combat ability visual effects (SG_Acometida, SG_CaprichoFloral, and others) for reasons complementary to the HLSL shaders: (1) visual iteration speed is superior for effects requiring frequent adjustment of UV distortion, emission, and transparency parameters; (2) native integration with the Particle System allows particle materials to use Shader Graph exposed properties as control variables; (3) VFX effects do not require GPU Instancing or the special compilation directives needed for geometry shaders, so there are no technical advantages to writing them in pure HLSL for this use case.

3.5 Blender

Blender was selected as the modeling and animation tool because it is free and open-source software with no licensing cost, making it sustainable for an independent development project. It offers a complete pipeline in a single tool: polygon modeling, UV

unwrapping, rigging, skinning, and animation editing, eliminating the need to transfer assets between separate applications. Its non-destructive modifier workflow allows geometry to be iterated without losing the base model state. Native FBX export guarantees direct compatibility with Unity without loss of hierarchies, normals, or animation data. In the project it was used both for original character modeling and for modifying external assets: retopology, scale adjustment, UV correction, and creation of the chest opening animation.

3.6 Mixamo

Animations for the protagonist character Rhaegal were obtained through Mixamo (Adobe), which provides an automatic rigging system and a library of animations free to use for educational and commercial projects. The process included uploading the character model for automatic rigging, selecting animations for idle, walk, attack, and take-damage states, and exporting in FBX format for import into Unity. Animations were integrated via an Animator Controller with boolean parameters and triggers that respond to CombatManager events.

3.7 GIMP

GIMP was used for texture processing and editing in the project. It provides the essential capabilities for a 2D texture pipeline — layers, masks, alpha channels, and non-destructive color adjustments — at no licensing cost. Its support for standard formats (PNG, TGA, PSD) maintains direct compatibility with Unity's texture importer. It was used primarily for adjusting the color palette of external textures, generating transparency maps, and correcting seamless tiling in environment materials.

4. Development Methodology

4.1 Iterative Incremental Methodology

The project followed an Iterative Incremental methodology, dividing development into short cycles where each iteration delivers a verifiable functional increment. This strategy was chosen because the project's components have differentiated natures — 3D art, systems programming, and visual pipeline — that can be partially developed in parallel

without blocking other areas. Visual requirements (shaders, lighting, post-processing) require frequent adjustment upon observing runtime results, so short iterations provide rapid visual feedback. The highest-risk technical components, such as custom shader development and combat logic, were addressed in intermediate phases to allow time for correction before closure.

4.2 Project Phases

Development was structured in four phases executed from February 8 to May 25, 2026.

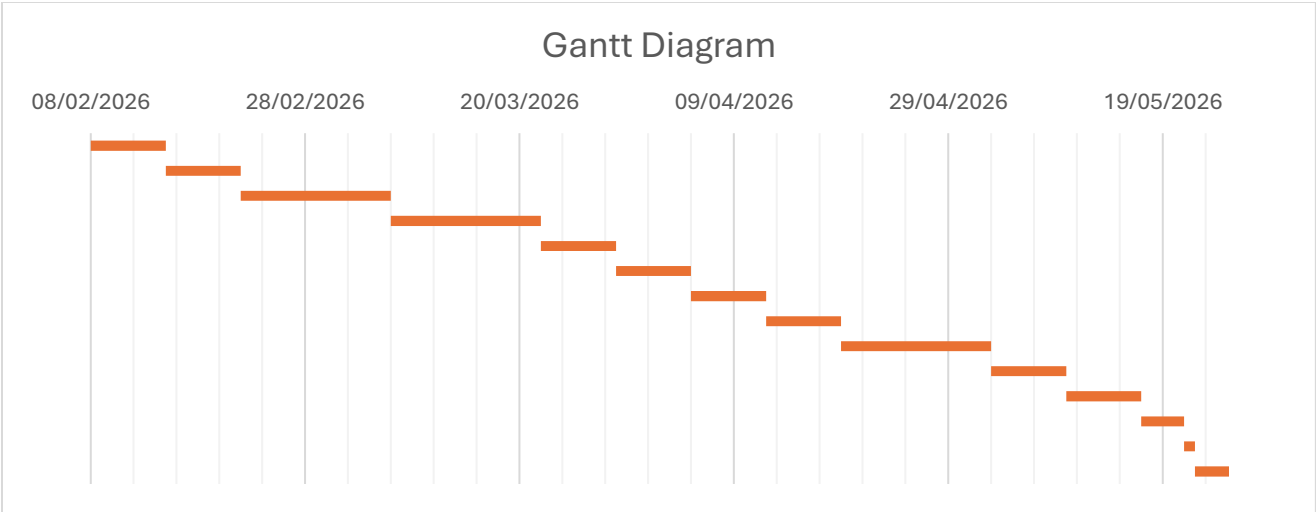
Phase 1 — Pre-production (02/08/2026 – 03/21/2026) Establishment of the project's conceptual, visual, and design foundations. Visual style references were researched to define the Dark Fantasy Toon art direction, followed by documentation of the prototype's objective, scope, and limitations. In parallel, the main characters were modeled in Blender and the exploration and combat scenes were designed alongside HUD prototyping.

Phase 2 — Implementation (03/22/2026 – 04/11/2026) Integration of produced assets into the Unity project and development of the core game systems. Models and textures were exported from Blender with UV and base material configuration, dungeon exploration logic was implemented, and the turn-based combat system was developed along with its action menu.

Phase 3 — Assets and Rigging (04/12/2026 – 05/09/2026) Production of the project's advanced visual elements. Cycle animations (idle and combat poses) were generated via Mixamo and adjusted in Unity; custom HLSL toon shaders were developed for the URP pipeline; and particle systems and Shader Graph effects were implemented for combat abilities.

Phase 4 — Closure (05/10/2026 – 05/25/2026) Final prototype refinement and delivery. Full post-processing was configured (Tonemapping, Bloom, Vignette, Color Adjustments, SSAO), performance optimization techniques were applied (Static Batching, GPU Instancing), and performance testing was conducted. Technical and user documentation was written in parallel with testing, with the final delivery on May 21 and presentation preparation through May 25.

4.3 Gantt Diagram



Phase	Activity	Start Date	Duration (Days)	End Date
Pre-production	Research, References, and Visual Style	02/08/2026	7	02/14/2026
	Planning: Objective, Scope, and Limitations	02/15/2026	7	02/21/2026
	3D Character Design	02/22/2026	14	03/07/2026
	Scene Design and HUD Prototyping	03/08/2026	14	03/21/2026
Implementation	Model and Texture Export	03/22/2026	7	03/28/2026
	Exploration Programming	03/29/2026	7	04/04/2026
	Turn Logic and Menu Programming	04/05/2026	7	04/11/2026
Assets Rigging &	Cycle Animation (Idle and Poses)	04/12/2026	7	04/18/2026
	Shader Development	04/19/2026	14	05/02/2026
	VFX and Particle Implementation	05/03/2026	7	05/09/2026
Closure	Lighting, Post-processing, and Polish	05/10/2026	7	05/16/2026
	Performance Testing and Documentation	05/17/2026	4	05/20/2026

Final Delivery	05/21/2026	1	05/21/2026
Presentation Preparation	05/22/2026	4	05/25/2026

5. Architecture and Specifications

5.1 System Architecture

The system is organized into four functional layers:

Data Layer (ScriptableObjects): CharacterData and Magic are ScriptableObjects that store character base statistics and ability parameters in a serializable, code-independent form. This architecture allows game balance modification without recompilation, facilitates configuration testing via the Unity Inspector, and separates design data from programming logic.

Game Logic Layer: CombatManager orchestrates the turn-based combat flow, managing action order, attack resolution (accuracy check via Random.Range, damage calculation accounting for target defense, area-of-effect application when esArea=true). GameManager persists across scenes via DontDestroyOnLoad and implements the Singleton pattern for global access.

Visual Presentation Layer: CombatAnimator coordinates character animation playback (Animator Controller with Mixamo clips), particle effect instantiation (targetEffectPrefab referenced in each Magic ScriptableObject), and audio clip playback (audioClip from Magic) synchronized with casting timings (tiempoLanzamiento, effectDuration).

Rendering Pipeline Layer: URP with ForwardRenderer manages scene rendering. Custom HLSL shaders execute in the Forward Pass. The Global Volume applies full-screen post-processing effects. PC_Renderer.asset includes the SSAO RendererFeature (Screen Space Ambient Occlusion).

5.2 Data System: ScriptableObjects

The project uses two ScriptableObject types defined in C# code (CharacterData and Magic):

CharacterData contains: nombre, descripcion, ataqueFisicoBase, poderMagico, defensaBase, vidaBase, velocidadBase, evasionBase, precisionBase, ataquesPorTurno, and an array of references to Magic ScriptableObjects. This structure allows defining complete characters as Unity assets editable in the Inspector.

Magic contains: nombre, descripcion, magicType (physical/magical), esArea (bool to affect all allies/enemies), valor (base damage/healing), precision (hit probability, 0–100), usosMaximos, castClip (AnimationClip), tiempoLanzamiento, projectilePrefab, targetEffectPrefab (VFX Prefab), effectDuration, and audioClip.

5.3 Combat System Flow

The combat flow follows this cycle per turn: (1) CombatManager determines the active character based on velocidadBase; (2) SubMenu presents available options (Attack, Magic, Defense, Item); (3) the player selects an action and target; (4) CombatManager calls CombatAnimator.PlayAbility(magic, caster, target); (5) CombatAnimator plays the castClip in the caster's Animator for tiempoLanzamiento seconds; (6) instantiates the targetEffectPrefab at the target's position; (7) waits effectDuration seconds for the VFX to complete its cycle; (8) CombatManager applies the damage/healing value adjusted by statistics; (9) checks the win or loss condition; (10) passes the turn to the next active character.

5.4 Lighting System

The scene uses two types of light sources in Realtime mode. A Directional Light acts as the scene's primary light with a warm white color, intensity 1, and rotation (X: 50°, Y: -30°) simulating a lateral overhead source, with Soft Shadows enabled at strength 1. The global shadow color is set to #6B7AA0, a cool blue-violet that creates complementary contrast with the warm tones of the point lights — a standard artistic lighting technique for enhancing the sense of depth. The Point Lights positioned on torches and candelabras complement the Directional Light via the FlickerAnimator component, which animates their intensity in a smoothed random pattern to simulate organic flame behavior. Environment Lighting and Environment Reflections use the scene's active Skybox as their source.

5.5 Materials System

The project's materials are divided into two categories based on complexity. Large, highly visible surfaces — such as the floor and ceiling — use full texture sets with albedo, normal map, displacement map, AO, and specular, providing high-frequency detail where the surface area visually justifies it. The remaining materials, which make up the majority of the dungeon and character assets, apply the custom toon shader with flat color palettes and no additional texturing. This decision is consistent with the project's low-poly style: faceted geometry and cel-shading make photorealistic textures visually inconsistent with the art direction, making flat color a deliberate production choice.

6. Technical Implementation Analysis

This section details the technical implementation of each project subsystem, including design decisions, configured parameters, and evaluated alternatives.

6.1 Custom Shader

Three HLSL shaders were developed in code (not Shader Graph) for the URP Forward pipeline: `Toon_Environment.shader` for environment geometry (walls, columns, floors, static objects), `Toon_General.shader` for objects with variable material properties, and `Toon_Rhaegal.shader` for the protagonist character with specific shading characteristics.

The implemented lighting model is toon/cel-shading based on the normalized dot product between the light vector and the surface normal ($N \cdot L$). Rather than applying the $N \cdot L$ value directly (continuous Lambertian diffuse), a quantization function groups the result into discrete bands, producing the characteristic cartoon shading with abrupt transitions between lit and shadowed areas.


```
float NdotL = dot(normalize(normalWS), normalize(lightDir));  
  
float toon = smoothstep(0.0, 0.02, NdotL);  
  
float3 diffuse = toon * lightColor * albedo;  
  
float spec = pow(max(0, dot(halfDir, normalWS)), 32.0);  
  
float toonSpec = step(0.5, spec);  
  
float3 specular = toonSpec * lightColor * 0.5;
```

Representative HLSL lighting calculation fragment (pseudocode):

GPU Instancing Support During the optimization phase, it was identified that the custom HLSL shaders lacked the compilation directives required for GPU Instancing. The following modifications were added to Toon_Environment.shader:

```
#pragma multi_compile_instancing  
  
UNITY_VERTEX_INPUT_INSTANCE_ID  
  
UNITY_SETUP_INSTANCE_ID(IN);
```

These additions allow Unity to batch instances of the same mesh+material into a single draw call when GPU Instancing is enabled on the material, resulting in an additional reduction of 18 draw calls on top of the Static Batching already applied.

Technique justification Cel-shading was selected for coherence with the project's Dark Fantasy Toon art direction. This style reduces dependency on high-resolution normal maps, leveraging the low polygon count geometry by making flat facets part of the visual style. The project's saturated color palette integrates naturally with the toon lighting model, which tends to exaggerate color transitions in ways favorable to visual readability.

6.2 Visual Effects (VFX)

Particle systems and VFX shaders for combat abilities.

Technical description Combat ability visual effects were implemented via two complementary mechanisms: particle systems (Unity/Shuriken Particle System) and Shader Graph materials assigned to particle emitters.

Each ability (Magic ScriptableObject) references a targetEffectPrefab containing a Particle System configured with the corresponding Shader Graph material. When CombatAnimator.PlayAbility() is invoked, the prefab is instantiated at the target's position, plays for effectDuration seconds, and is automatically destroyed. This architecture decouples the effect definition (ScriptableObject) from its visual presentation (Prefab), allowing new abilities to be added without modifying combat code.

The Shader Graph effects developed for VFX include exposed parameters such as Dissolve Amount, Emission Intensity, and UV Scroll Speed, allowing the particle system to interpolate these values over the particle lifetime via the Renderer module and Custom Vertex Streams.

Technique justification Unity's Particle System was chosen over VFX Graph because it has native support in the URP Forward renderer without additional configuration, is compatible with all of the project's platform targets, and its behavior is deterministic in the context of a turn-based game where effects must complete in timings synchronized with combat logic. VFX Graph, being GPU compute-based, would have required additional compatibility adjustments with the URP Forward renderer in Unity 6.

6.3 Technical Lighting

Directional Light as the primary source and dynamic Point Lights with FlickerAnimator on torches and candelabras.

Technical description The DungeonExploration scene uses real-time dynamic lighting composed of two source types. A Directional Light acts as the primary light with a warm white color, intensity 1, and angle (X: 50°, Y: -30°) simulating lateral overhead illumination with Soft Shadows enabled. The global shadow color is set to #6B7AA0, a cool blue-violet that contrasts with the warm tones of the point lights, generating depth through complementary contrast.

The Point Lights positioned on torches and candelabras have the FlickerAnimator component assigned, which animates their intensity following a smoothed random curve to simulate the organic behavior of a flame. Their colors in the orange-amber range reinforce the dungeon atmosphere and contrast with the environment's global shadow color.

Static vs. Dynamic Lighting Decision Light Baking (Lightmapping) was evaluated for performance improvement but was discarded for the following reasons: Point Lights with FlickerAnimator are dynamic and cannot be included in a static lightmap without producing visual inconsistencies between the baked base light and the realtime flicker; the player character would require Light Probes to receive illumination from the lightmap, adding configuration complexity; and fully dynamic lighting produces satisfactory visual results with the toon shader without the overhead of the bake process.

Occlusion Culling Decision Occlusion Culling was evaluated using Unity's integrated system but was discarded because the DungeonExploration scene has open-box geometry with direct visibility to all objects from any character position. In this configuration, no effective occluders would be generated, and the bake processing cost and runtime verification overhead would exceed any performance benefit.

6.4 Post-Processing

URP Global Volume with five configured effects.

Technical description Post-processing is implemented via a URP Global Volume referencing the SampleSceneProfile.asset profile, shared by both DungeonExploration and Combat_01 to ensure visual consistency between them. The Global Volume operates in Global mode, applying its effects at all times without proximity conditions.

Tonemapping (Neutral): maps the HDR range of the render buffer to the LDR display range. Neutral mode preserves the color saturation of the toon shaders better than ACES (which introduces a warm tonal shift) or Filmic (which compresses highlights non-linearly), producing the most faithful representation of the color values defined in the materials.

Bloom (Threshold: 0.6, Intensity: 1.0, Scatter: 0.4): generates a soft glow halo around pixels exceeding the luminosity threshold. With threshold 0.6 the effect primarily activates

on direct light sources and toon shader specular highlights. Scatter 0.4 controls the halo extension without making it invasive.

Vignette (Intensity: 0.35, Smoothness: 0.4, Rounded: true): darkens the screen edges, concentrating attention on the center of the frame. Rounded mode produces an optically more natural vignette than the default rectangular variant.

Color Adjustments (Contrast: +15, Saturation: +20): the contrast increase complements the toon shader quantization by emphasizing the difference between lit and shadowed areas. The saturation increase intensifies the warm oranges of the torches and the cool blues of the shadows, accentuating the dark fantasy palette without producing colors that break artistic coherence.

Screen Space Ambient Occlusion (Intensity: 0.4): implemented as a `RendererFeature` in `PC_Renderer.asset`. Calculates ambient occlusion in screen space, darkening corners, crevices, and surface joints. With intensity 0.4 it adds depth without producing halo artifacts at edges — a frequent problem in low-poly geometry when intensity exceeds 0.6.

6.5 Optimization and Performance

Static Batching

Static Batching combines at build time the meshes of objects marked as Static into a single combined vertex buffer, eliminating the draw call overhead per individual object.

Objects marked as Static in `DungeonExploration` include walls, columns, floors, ceilings, stairs, pedestals, and non-animated decorative props. Lights with `FlickerAnimator` and collectible objects were excluded because they are dynamic at runtime.

Result measured in Unity's Stats Panel: 170 batches before → 105 batches after, a reduction of 65 batches (38.2%), verified via Frame Debugger.

GPU Instancing

GPU Instancing allows rendering multiple instances of the same mesh+material in a single draw call, passing individual transforms as a data buffer on the GPU.

Enable GPU Instancing was checked on the 18 scene materials using Toon_Environment. Additionally, explicit support had to be added to the HLSL shader — unlike Shader Graph, code-written shaders require the compilation directives manually: `#pragma multi_compile_instancing`, `UNITY_VERTEX_INPUT_INSTANCE_ID` in the `Attributes` struct, and `UNITY_SETUP_INSTANCE_ID(IN)` as the first instruction of the vertex shader.

Evaluated and Discarded Techniques Object Pooling: discarded because the VFX instantiation frequency in a turn-based RPG (1–2 effects every 2–5 seconds) is well below the threshold where pooling generates measurable benefit.

Level of Detail (LOD): discarded because the fixed isometric camera keeps all objects at similar distances, with no on-screen size variation that would justify multiple detail levels.

Lightmapping: discarded due to incompatibility with dynamic `FlickerAnimator` light sources, detailed in section 6.3.

6.6 System Integration

Coherent integration between the data system, combat logic, animations, VFX, and audio.

Technical description System integration manifests in the complete execution chain of an ability: the `Magic ScriptableObject` contains references to all resources required for its execution (`AnimationClip`, `VFX Prefab`, `AudioClip`) without coupling to specific classes. `CombatManager` consumes the `ScriptableObject` data for logic decisions (damage, area of effect, accuracy) and delegates presentation to `CombatAnimator`, which coordinates the temporal sequence of animation, VFX, and audio.

`GameManager` implements the Singleton pattern with `DontDestroyOnLoad`, persisting across scenes (Menu > DungeonExploration > Combat_01 > Menu) and keeping game state coherent across transitions. `SaveSystem` uses `PlayerPrefs` to serialize state without external dependencies.

Visual pipeline coherence The `tiempoLanzamiento` from the `Magic ScriptableObject` defines when the VFX is instantiated, synchronized with the culminating frame of the attack animation. The `effectDuration` defines when `CombatManager` applies the damage,

synchronized with the visual climax of the effect. This temporal coordination produces a cohesive experience where visual presentation and game mechanics are aligned.

6.7 Animation System

Character animations are managed via Animator Controllers in Unity with clips imported from Mixamo in FBX format. Each character has defined animation states (Idle, Walk, Attack, TakeDamage) connected via transitions conditioned by boolean parameters and triggers. CombatAnimator is the component responsible for activating these parameters in response to CombatManager events, ensuring that the attack animation executes before VFX instantiation and that the damage animation responds to combat resolution. Transition times between states are tuned to minimize abrupt visual cuts without introducing perceptible delays in combat response.

6.8 Audio System

Audio operates at two levels. Background music plays via an AudioSource in loop assigned to the GameManager, allowing it to persist across scenes without interruption during transitions. Ability sound effects are referenced directly in each Magic ScriptableObject via the audioClip field, allowing CombatAnimator to play them synchronized with the corresponding animation and VFX without requiring centralized audio logic. SFX clips play in 2D mode since the camera is fixed and the character's on-screen position does not vary enough to justify positional audio with spatial attenuation.

7. Asset Management and Credits

The following assets were used in the project. All external assets were modified or adapted to integrate with the prototype's visual and technical pipeline.

Asset	Platform	Author(s)	Type	Applied Modifications
Combat SFX	Freesound.org	nicksta18, eldiariosonoro_, cjosephwalker, SilverIllusionist, steprock, colorsCrimsonTea rs, gezortenplotz,	Audio (SFX)	Level normalization, silence trimming, format adjustment for Unity AudioSource

NoahBangs, Coghezzi, magnuswaker					
BGM	peritune.com	Asset	Peritune	Audio (Music)	Loop configured in Unity AudioSource, volume adjustment relative to SFX
Inferno World Free	Unity Store	Asset	ithappy	3D Environme nt	Shaders replaced with custom Toon_Environment.sha der
Low Poly Dungeons Lite	Unity Store	Asset	JustCreate	Dungeon geometry	Blender retopology, shader replacement, chest opening animation added
Low Poly RPG Bundle Part 3	Unity Store	Asset	GualtierisGG	Props and characters	Proportion modification in Blender, toon shaders applied, UV adjustment
Surreal- Skyboxes	Unity Store	Asset	igor-lir	Skybox	Variant selection for dark fantasy atmosphere
Combat backgrou nd	Vecteezy.co m		—	2D Image	Saturation and contrast adjustment for consistency with game palette
Character animation s	Mixamo		Adobe Inc.	FBX Animations	Automatic rigging, idle/attack/damage clip selection, Animator Controller setup
Main characters	Original production		—	3D Models (Blender)	Original modeling, retopology, UV unwrapping, rigging preparation
Game engine	Unity Technologie s		—	Software	N/A

8. Cost Analysis

8.1 Labor Costs

The project was developed by one team member assuming multiple technical roles. Hour estimates are based on activities recorded in the Gantt diagram, with a market reference rate for a junior video game development and technical art profile.

Role	Estimated Hours	Rate (MXN/hour)	Total (MXN)
3D Artist (modeling, rigging, animation)	180	\$150	\$27,000
Developer (C#, game systems)	160	\$150	\$24,000
Technical Artist (shaders, VFX, post-processing)	100	\$150	\$15,000
Design and documentation	60	\$150	\$9,000
Total	500		\$75,000

8.2 Software Licenses

Software	License	Justification	Cost (MXN)
Unity 6	Personal (free)	The Personal license is free for projects with annual revenue below \$200,000 USD. A prototype of this scale does not exceed that threshold.	\$0
Blender 4.x	Open Source (GNU GPL)	Free software with no commercial use restrictions.	\$0
GIMP	Open Source (GNU GPL)	Free software with no commercial use restrictions.	\$0
Mixamo	Free plan (Adobe)	Free for personal and commercial use with unlimited downloads.	\$0
JetBrains Rider	Individual commercial	A free student license was used during development. The commercial reference cost is \$69 USD/year (~\$1,173 MXN at the current exchange rate).	\$1,173
Total			\$1,173

8.3 Hardware and Depreciation

Component		Specification		Estimated Price (MXN)	Useful Life	Monthly Depreciation	Months of Use	Total (MXN)
HP 15.6	Victus	Ryzen 7525HS, 32GB DDR5, RTX 2050	5	\$18,999	36 months	\$527.75	3.5	\$1,847

8.4 Operating Costs

Item	Detail	Cost (MXN)
Electricity	500h × 100W average = 50 kWh × \$2.50 MXN/kWh	\$125
Internet	\$450 MXN/month × 3.5 months (proportional to project)	\$1,575
Total		\$1,700

8.5 Cost Summary

Item	Cost (MXN)
Labor	\$75,000
Software	\$1,173
Hardware (depreciation)	\$1,847
Operating costs	\$1,700
Project Total	\$79,720

9. Technical Dictionary

9.1 Core System

Script	Purpose	Key Methods	Key Variables
AudioManager.cs	Persistent singleton.	PlayBGM(clip), PlaySFX(clip),	bgmSource, sfxSource,

	Manages BGM and SFX across the entire game, automatically switching music when scene changes are detected.	PlayBGMFin(), StopBGM(), PlayBoton(), PlayAtaqueBasico(), PlayEvilLaugh()	bgmMenu, bgmExploracion, bgmCombate, bgmFin, sfxAtaqueBasico, sfxRecibirDano, sfxMuerte
PartyManager.cs	Persistent singleton. Stores and synchronizes party state across scenes.	LoadIntoStats(partyStats[]), SaveFromStats(partyStats[])	characterDatas[], characterSaveDatas (List)
SaveSystem.cs	Manages serialization and deserialization of game state via PlayerPrefs.	Save(data), Load(), HasSave(), DeleteSave()	SAVE_KEY (constant)
ExplorationLoaders	Restores player position and inventory when loading the exploration scene from an existing save.	Start()	playerTransform
MainMenuManagers	Controls main menu buttons. Enables or disables "Continue" based on whether a save exists.	NuevoJuego(), Continuar()	botonContinuar, escenaExploracion

PauseMenuManager.cs	Manages the pause menu in exploration. Saves the game when paused.	Pausar(), Reanudar()	pauseMenu, playerTransform, pausado
----------------------------	--	----------------------	-------------------------------------

9.2 Data (ScriptableObjects)

Script	Purpose	Key Methods	Key Variables
CharacterData.cs	Defines a character's base statistics and ability list. Instantiable from the Inspector without code.	—	nombre, ataqueFisicoBase, poderMagico, defensaBase, vidaBase, velocidadBase, evasionBase, precisionBase, ataquesPorTurno, magias[]
Magic.cs	Defines an ability's parameters: type, value, accuracy, and references to visual and audio resources.	—	nombre, magicType, esArea, valor, precision, usosMaximos, castClip, tiempoLanzamiento, targetEffectPrefa

		b, effectDuration, audioClip
ItemData.cs	Defines an inventory — item's data: name, description, and healing amount applied.	nombre, descripcion, healAmount
Item.cs	Runtime instance of an Item(itemData, ItemData with quantity. quantity) Used by the inventory system.	ItemData, ItemQuantity, nombre, descripcion
CharacterSaveData.cs	Serializable structure that stores a character's persistent state across scenes.	CharacterSaveData(characterData, currentLv, currentXp, currentHealth, hasBeenSaved

9.3 Combat System

Script	Purpose	Key Methods	Key Variables
CombatManager.cs	Orchestrates the turn-based combat flow. Manages selection and execution phases, and win/loss conditions.	SelectAction(action), ElegirObjetivoAliado(states), EjecutarTurnos()	partyMembers[], enemy, currentState, turnData[], currentPartyIndex, esperandoObjetivo
CombatCalculator.cs	Static class with the combat system's mathematical formulas. Separated	Precision(precision, evasion, turnData), ActionValue(emitter, receptor, turnData)	— (stateless static class)

from the Manager to keep calculation logic isolated.

CombatAnimator.cs	Coordinates the temporal sequence of animation, VFX, and audio for each combat action.	Init(party, enemy), PlayAbility(magic, caster, target), PlayAttack(caster, target)	partyAnimators[], enemyAnimator, effectDuration
CharacterAnimator.cs	Wraps a character's Animator and exposes coroutines synchronized with Mixamo clip timings.	PlayAttackUntilLaunch(), , PlayAttackRemainder(), PlayMagicUntilLaunch(magic), PlayMagicRemainder(magic), PlayDefend(), PlayTakeHit(), PlayDie(), SetMagicClip(clip), SetWalking(bool)	duracionAtaque, duracionMagia, tiempoLanzamiento, puntoLanzamiento, puntoImpacto, magicPlaceholderClip Name
ActionMenu.cs	Manages the main combat action menu (Attack, Magic, Defense, Item). Disables buttons while waiting for target selection.	AtackBtn_Click(), MagicBtn_Click(), DefenseBtn_Click(), ObjectBtn_Click()	atackBtn, defenseBtn, magicBtn, objectBtn, subMenu, combatManager
SubMenu.cs	Presents contextual submenus (ability list, item list) when an action is selected in ActionMenu.	ShowMagicMenu(), ShowItemMenu(), HideSubMenu()	combatManager, magicButtons[], itemButtons[]

CharacterSelect.cs	Allows selecting an ally as a target by clicking on their on-screen representation.	OnPointerClick(eventData), OnPointerEnter(eventData), OnPointerExit(eventData)	misEstadisticas, combatManager, marcoVisual
CombatLoggers	Combat log system with typewriter effect and message queue. CombatManager waits for IsIdle before advancing phases.	Log(message), LogSeleccion(nombre), LogPhase(phase)	combatDialogue, letraporSegundo, tiempoEntresMensajes, cola (Queue), IsIdle
EndScreen.cs	End-of-game screen (victory or defeat) with fade and input wait to return to the menu.	Mostrar(victoria)	panel, mensaje, textoContinuar, escenaMenu

9.4 Exploration System

Script	Purpose	Key Methods	Key Variables
PlayerController.cs	Controls player movement via Rigidbody, walking and item pickup with the E key.	RecogerItem(item), FixedUpdate(), Update()	speed, rotationSpeed, characterAnimator, nearbyItem, intervaloPasos
CheckBounds.cs	Restricts the player's position within the floor BoxCollider's boundaries.	Update()	floor, offset, xMax, xMin, zMax, zMin

CameraFollow.cs	Follows the player with smooth interpolation and transitions between exploration and combat camera configurations.	EventStart(), EventEnd()	offsetX, offsetY, offsetZ, velocidadMov, combatFOV, exploreRotation, combatRotation
CombatStart.cs	Manages the combat start sequence on trigger zone contact: moves the enemy, fires dramatic audio, and loads the combat scene.	IniciarCombate(), SmoothLookAt(looker, target, duration)	player, enemy, enemyTarget, cameraFollow, speed, pausaDramatica, walkAnimSpeed
ChestManager.cs	Manages interaction with the treasure chest: triggering the opening animation and delivering the reward.	OnInteract(), OpenChest(), GiveReward()	isOpen, animatorRef, reward
Inventory.cs	Persistent singleton. Manages the player's global inventory with item stacking support.	AddItem(item), RemoveItem(item), SearchItem(item)	ItemsList (List)
CollectableItem.cs	Component assigned to collectible scene objects. Contains a reference to the corresponding ItemData.	—	itemData

9.5 HUD / Interface

Script	Purpose	Key Methods	Key Variables
CharacterStats.cs	Manages character's dynamic in-game statistics: current HP, level, experience, and derived stat calculation.	TakeDamage(damage), ReceiveHeal(amount), RecalculStats(), RefreshHealthUI(), StatLvUpdate(baseStat, nivel)	currentHealth, currentMaxHealth, currentLv, currentXp, characterData, healthBar, xpBar, StatScale

HealthBarSystem.cs	Smoothly animates the health bar toward a target value using <code>Mathf.MoveTowards</code> .	<code>SetHealth(percentage)</code>	<code>healthBar (Image)</code> , <code>barSpeed</code> , <code>nextHealth</code>
XpBarSystem.cs	Smoothly animates the experience bar. Visually resets on level-up.	<code>SetXp(percentage)</code> , <code>LevelUp(level)</code>	<code>xpBar (Image)</code> , <code>barSpeed</code> , <code>nextXp</code>
PickupNotifier.cs	Singleton that manages a stack of toast notifications when items are picked up. Toasts appear stacked and float upward.	<code>Notify(texto)</code>	<code>toastPrefab</code> , <code>toastContainer</code> , <code>toastSpacing</code> , <code>toastDuration</code> , <code>activeToasts (List)</code>
PickupToast.cs	An individual pickup toast. Floats upward and fades out during its lifetime.	<code>SetText(texto)</code> , <code>Animator(duration, risePerSecond)</code>	<code>canvasGroup</code> , <code>rectTransform</code>

9.6 VFX

Script	Purpose	Key Methods	Key Variables
FlickerAnimator.cs	Animates a Point Light's intensity with smoothed random variation to simulate organic flame.	<code>Update()</code>	<code>minIntensity</code> , <code>maxIntensity</code> , <code>flickerSpeed</code> , <code>lightRef</code>
Billboard.cs	Rotates the Transform to always face the camera. Used on the torch flame quad.	<code>LateUpdate()</code>	<code>copyRotation</code>
DissolveAnimator.cs	Animates the <code>_DissolveAmount</code> property of a VFX effect's material: fade-in on spawn and fade-out before destruction.	<code>Start()</code> , <code>Update()</code>	<code>lifetime</code> , <code>fadeIn</code> , <code>fadeOut</code> , <code>dissolveProperty</code> , <code>disappearProperty</code>

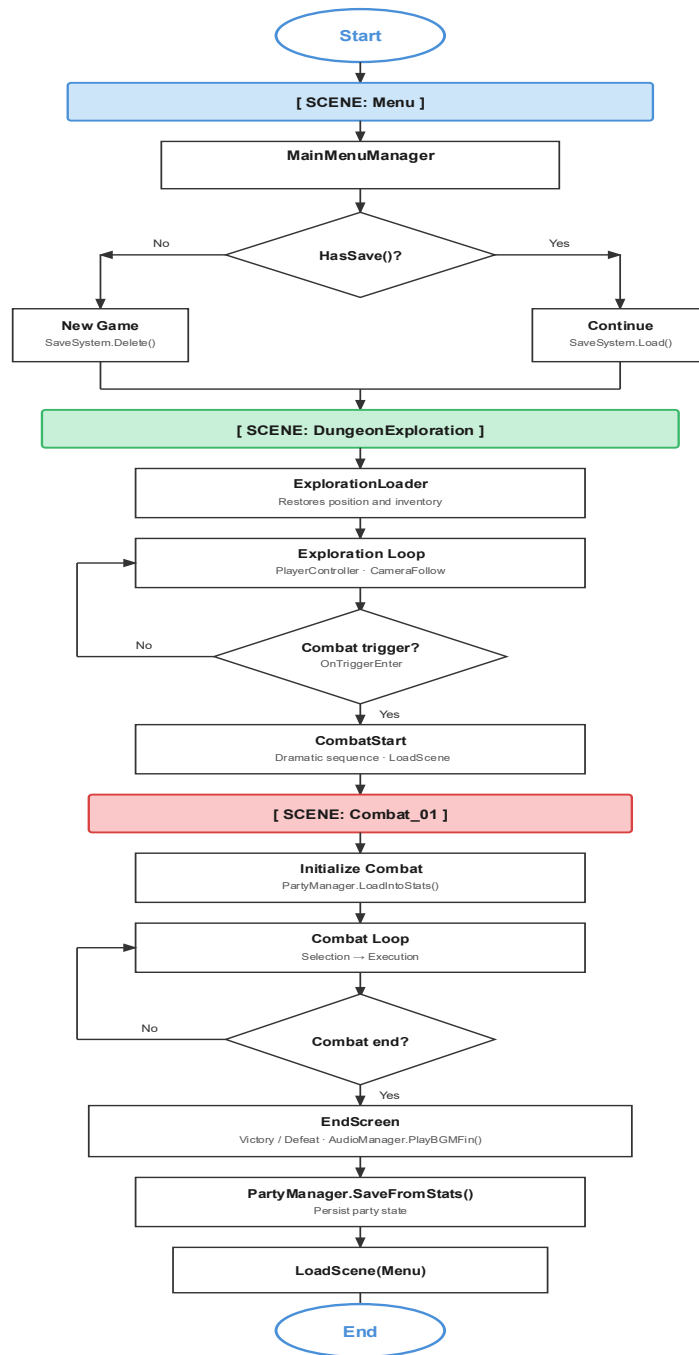
BeamConnector.cs	Visually connects two points (caster and target) via a continuous beam effect.	Connect(origin, target), Disconnect()	lineRenderer, origin, target
-------------------------	--	--	---------------------------------

Projectile.cs	Controls the movement of projectiles instantiated by combat abilities with a projectile-type targetEffectPrefab.	Launch(target), Update()	speed, targetPosition, hasHit
----------------------	--	-----------------------------	-------------------------------------

10. Troubleshooting

Problem	Symptoms	Cause	Solution
GPU Instancing inactive after enabling checkbox	Frame Debugger showed individual draw calls per instance despite Enable GPU Instancing being checked on materials	Custom HLSL shader (Toon_Environment) lacked the compilation directives required by GPU Instancing	Add <code>#pragma multi_compile_instancing, UNITY_VERTEX_INPUT_INSTANCE_ID</code> in Attributes, and <code>UNITY_SETUP_INSTANCE_ID(IN)</code> in vertex shader
Low performance at 4K resolution during development	Framerate < 30 FPS in Play mode with Game View resolution at 3840×2160	Post-processing (especially SSAO and Bloom) scales quadratically with screen resolution	Set Game View to 1920×1080 during development. Final build resolution is independent of the editor Game View.

11. Flow Diagrams



12. Conclusions

Beyond the Firmament establishes a functional technical foundation that coherently integrates six rendering and VFX subsystems within a turn-based RPG prototype. The project demonstrates that Unity 6 with URP is a viable platform for independent production with a defined artistic visual identity, allowing custom HLSL shaders, particle systems, dynamic lighting, and full-screen post-processing to be combined without compromising the game architecture.

The most significant technical decisions of the project — fully dynamic lighting justified by the incompatibility of baking with FlickerAnimator, discarding Occlusion Culling due to open-box geometry, and the manual addition of GPU Instancing support to HLSL shaders — reflect a technical evaluation process that goes beyond the mechanical application of techniques, considering the real impact of each decision on the existing architecture and visual result.

Future work lines identified include: VFX Graph implementation for computationally complex effects, a JSON serialization-based save system for multiple save slot support, facial animation integration via blend shapes, and pipeline preparation for resolutions above 1080p through post-processing optimization.

13. Bibliografía / Bibliography

- Adobe Inc. (2024). *Mixamo* [Servicio de animación 3D / 3D animation service]. <https://www.mixamo.com>
- Blender Foundation. (2024). *Blender 4.x* [Software]. <https://www.blender.org>
- GualtierisGG. (s.f. / n.d.). *Low Poly RPG Bundle Part 3* [Asset pack]. Unity Asset Store. <https://assetstore.unity.com>
- ithappy. (s.f. / n.d.). *Inferno World Free* [Asset pack]. Unity Asset Store. <https://assetstore.unity.com>
- igor-lir. (s.f. / n.d.). *Surreal-Skyboxes* [Asset pack]. Unity Asset Store. <https://assetstore.unity.com>
- JustCreate. (s.f. / n.d.). *Low Poly Dungeons Lite* [Asset pack]. Unity Asset Store. <https://assetstore.unity.com>
- nicksta18, eldiariosonoro_, cjosephwalker, SilverIllusionist, steprock, colorsCrimsonTears, gezortenplotz, NoahBangs, Coghezzi, & magnuswaker. (s.f.

/ n.d.). *Sound effects* [Archivos de audio / Audio files]. Freesound.
<https://freesound.org>

- PeriTune. (s.f. / n.d.). *Música libre de derechos / Royalty-free music* [Archivos de audio / Audio files]. PeriTune. <https://peritune.com/>
- Unity Technologies. (2024). *GPU instancing*. Unity Documentation. <https://docs.unity3d.com/Manual/GPUInstancing.html>
- Unity Technologies. (2024). *Post-processing in URP*. Unity Documentation. <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.0/manual/EffectList.html>
- Unity Technologies. (2024). *Static batching*. Unity Documentation. <https://docs.unity3d.com/Manual/DrawCallBatching.html>
- Vecteezy. (s.f. / n.d.). *Fantasy backgrounds* [Ilustración digital / Digital illustration]. <https://www.vecteezy.com>